

SparseTreePIR: Active-Interval Coloring for Batch Private Information Retrieval of Sparse Merkle Proofs

Anonymous Author(s)

Abstract—Sparse Merkle Trees (SMTs) support compact membership proofs over large logical address spaces by combining real sibling digests with public default-empty hashes. A direct proof request, however, reveals the target coordinate and its authenticated path. We study how to organize fixed-snapshot SMT membership proof material as a target-private batch-PIR workload. The problem is not to design a new PIR primitive, but to identify which non-default SMT proof records should populate the PIR database and how they should be partitioned into PIR subdatabases. SparseTreePIR places only active proof-bearing sibling nodes in the PIR database, indexes them by served target-leaf intervals, colors the resulting interval forest, and exposes public metadata for fixed-shape real/dummy batch queries. The client retrieves at most one real node per color and completes the standard height- h SMT proof from the public default-empty hash chain. This separates the verifier-facing proof length from the PIR-facing active proof structure: the number of PIR subqueries is determined by non-default proof material rather than by every logical SMT level. We then introduce ActiveBalance, a subtree-load refinement that preserves the active-color query pattern while flattening the color subdatabases and reporting an instance-wise balance certificate. Experiments on real SMT workloads compare SparseTreePIR with privacy-preserving layouts derived from perfected TreePIR, pruned TreePIR, an optimistic PBC-style SMT baseline, and flat active PIR; SimplePIR and PIANO integrations further show that the resulting color stores can be queried by existing PIR implementations.

Index Terms—sparse Merkle tree, private information retrieval, batch PIR, authenticated map, proof retrieval

I. INTRODUCTION

Sparse Merkle Trees (SMTs) are Merkle trees over large logical address spaces. A height- h SMT has 2^h possible leaf coordinates, while typically materializing only a small subset of occupied leaves; empty subtrees are represented by publicly known default digests. This default-completion property makes SMTs a common building block for revocation maps, authenticated dictionaries, and blockchain state commitments [11], [14]–[17]. Despite the sparse representation, membership verification retains the standard Merkle interface: a verifier checks a leaf value together with one sibling digest per level and recomputes the committed root.

The standard SMT proof-serving interface is efficient, but it leaks the queried target. A server that receives a proof request learns which key, account, certificate, or state object the client is checking, and it also observes the corresponding authenticated path. Existing SMT work primarily optimizes storage layouts, proof generation, multiproofs, and update procedures. These optimizations reduce proof-serving cost, but

they do not hide the target of the proof request. This paper studies the complementary problem of *target-private retrieval* of the non-publicly reconstructible proof material for a fixed SMT snapshot.

Private information retrieval (PIR) is the natural primitive for hiding a database index, but an SMT proof is not a single database record. A standard proof is a height- h array containing two different types of entries: real sibling digests that must be retrieved from the server, and default-empty digests that the client can reconstruct locally. Querying each level independently through PIR would hide the target, but it would turn one structured proof request into many unrelated private lookups and would ignore the path structure of Merkle authentication.

Thus, private SMT proof retrieval is more naturally viewed as a batch-PIR workload: a client must retrieve several proof records using a fixed-shape transcript, without revealing which authenticated path those records instantiate. Generic batch-PIR placements can hide multiple requested indices, but they model the requested items as an arbitrary set. This line includes classical batch codes and combinatorial batch codes [29], [30], probabilistic batch codes (PBCs) as used in SealPIR [33], and recent multi-query PIR systems such as Vectorized BatchPIR and PIRANA [39], [40]. These systems improve the cryptographic or coding layer, but they do not identify which SMT-specific proof objects should populate the PIR database, nor do they specify how public default levels should be completed after private retrieval.

Merkle proofs have additional structure: the requested sibling digests are coupled by one root-to-leaf path. TreePIR [22] exploits this structure for perfect binary Merkle trees by coloring proof nodes so that every proof path contains at most one node of each color, then issuing one PIR subquery per color. This suggests the relevant structural principle for private proof retrieval: organize the proof database so that one fixed-shape batch query suffices for every target.

However, TreePIR’s perfect-tree abstraction does not directly match sparse SMT proof retrieval. Its coloring and indexing guarantees are developed for a full perfect-tree proof layout. If an SMT is first perfected, public default coordinates and inactive logical positions become part of the PIR-facing database, yielding a workload profile that is mismatched to sparse proofs. TreePIR discusses sparse Merkle trees as a possible extension, but the compressed sparse setting changes the object being colored and leaves the corresponding

indexing, metadata, and completion interface unresolved.

This is not a matter of applying TreePIR and deleting empty nodes afterward. Once public default material is removed from the PIR database, the remaining proof records no longer form the perfect-tree object for which TreePIR’s indexing argument is stated. One must identify which non-default sibling nodes are legitimate retrieval records, define the target set served by each record, publish enough metadata for private lookup without revealing the target, preserve a fixed-shape real/dummy transcript, and reconstruct the original height- h verifier proof from a partial private response. These obligations are absent from the perfect-tree model and are the technical content of the SMT-specific retrieval layer.

The key question is therefore not whether the one-query-per-color principle can be reused, but *what object it should be applied to*. SparseTreePIR answers this question by separating the verifier-facing proof from the PIR-facing database. The verifier still receives a standard height- h SMT membership proof. The PIR-facing database, however, contains only *active proof-bearing sibling nodes*: non-empty sibling-side nodes that can appear in membership proofs of occupied leaves.

This active structure is more than a storage filter. Each active node serves a contiguous interval of target leaves, and the collection of served intervals forms a laminar active-interval forest. SparseTreePIR colors this forest under an ancestor-distinct constraint, publishes served-interval metadata for target lookup, and uses a fixed-shape online transcript with one PIR subquery per active color. For a given target, a color slot is real if it contains the active sibling digest serving that target, and dummy otherwise. After retrieval, the client places the real outputs into the height- h proof array and fills all remaining levels using the public default-hash chain. Thus the proof semantics remain unchanged, while only the non-default proof material is retrieved through PIR.

This formulation also makes the relevant cost comparison explicit. Once target privacy is required, the baseline is not a non-private SMT proof API, but alternative privacy-preserving layouts that expose different database profiles to the PIR backend. The main backend-facing quantities are the number of PIR-facing proof records, the active batch width, the largest searched color store, and the public metadata size. After the active proof records have been identified, a second layout issue remains: a valid coloring may place substantially more records in one color store than another. SparseTreePIR addresses this with ActiveBalance, a subtree-load refinement that preserves the one-query-per-color query pattern while flattening the resulting color stores.

This paper makes the following contributions.

- 1) **PIR-facing proof-record abstraction.** We identify the active proof-bearing interval forest as the PIR-facing retrieval object for fixed-snapshot SMT membership proofs. This is the object missing from a direct sparse extension of TreePIR: the construction stores only non-default sibling digests as PIR records, publishes served-interval metadata for compact lookup, and reconstructs the standard height- h proof by local default-hash completion.
- 2) **Active-width fixed-shape retrieval.** We characterize the active width of the one-query-per-color SMT model and show that it is governed by the occupied-leaf skeleton rather than by the logical tree height. This yields a fixed-shape real/dummy batch-PIR transcript in which each active color contributes at most one real proof record.
- 3) **Balanced private-retrieval layout and evaluation.** We introduce ActiveBalance, a subtree-load refinement with an instance-wise balance certificate. Experiments on real SMT workloads compare SparseTreePIR with privacy-preserving layouts derived from perfected TreePIR, pruned TreePIR, an optimistic PBC-style SMT baseline, and flat active PIR. Repeated executable-backend runs with SimplePIR and PIANO show $2.1\times$ – $2.3\times$ lower online communication than the PBC-style baseline on the same real snapshots.

Figure 1 gives a compact map from ordinary SMT proof serving to the SparseTreePIR retrieval layout.

The rest of the paper is organized as follows. Section II gives the background and problem setting needed for our formulation. Section III positions SparseTreePIR relative to authenticated maps, TreePIR, and PIR backends. Section IV defines the problem, leakage model, and active proof-bearing nodes. Section V presents the SparseTreePIR construction. Section VI proves correctness, target privacy, and complexity properties. Section VII evaluates the resulting private-retrieval layout and balancing algorithm. Section VIII concludes.

II. BACKGROUND AND PROBLEM SETTING

This section fixes the terminology and retrieval model used throughout the paper. We focus only on the components that affect private proof retrieval: a Merkle proof is a structured batch of sibling digests; an SMT membership proof contains both non-default real sibling digests and publicly reconstructible default digests; and a PIR-based proof-retrieval scheme must expose a fixed-shape workload to the PIR backend without changing the verifier-facing proof semantics. The key distinction is between the proof seen by the verifier and the database seen by PIR: the former remains a standard height- h SMT proof, whereas the latter should contain only the proof material that cannot be derived from public default hashes.

A. Merkle Proof Retrieval

A binary Merkle tree commits to a set of records by hashing the records at the leaves and recursively hashing pairs of child digests until a single root digest is obtained [1]. To verify membership of a target leaf, a client uses the leaf value together with one sibling digest at each level on the path from the leaf to the root. The verifier recomputes the path upward and accepts if the reconstructed root matches the public commitment.

From the retrieval perspective, a Merkle proof is not a single database item. It is a structured batch whose entries are determined by one authenticated path. A direct proof-serving API therefore leaks the target leaf: when the client requests the proof for leaf i , the server learns both i and the corresponding

SparseTreePIR: Solving the SMT-to-Private-Retrieval Challenge

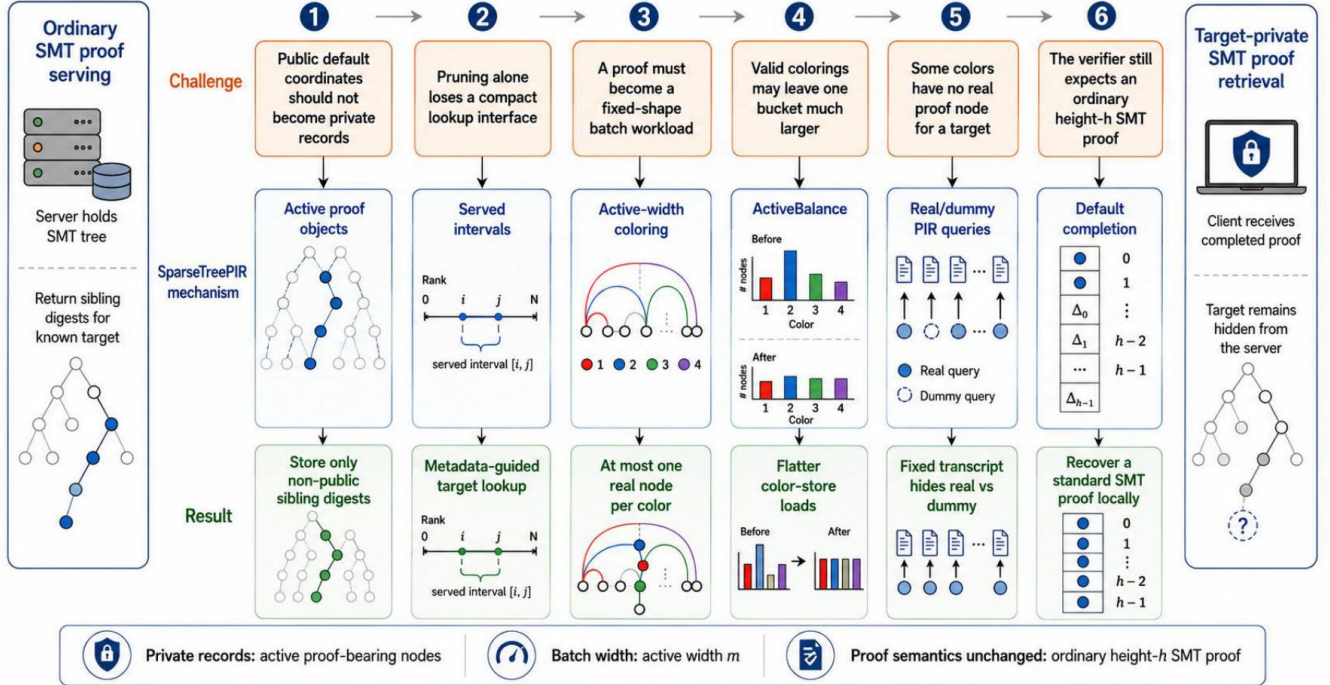


Fig. 1. SparseTreePIR solution map from ordinary SMT proof serving to target-private proof retrieval. The PIR-facing mechanism places only active proof records in color stores, uses served intervals and active-width coloring for fixed-shape batch retrieval, balances stores with ActiveBalance, hides real and dummy PIR roles, and recovers the standard height- h SMT proof locally.

proof path. TreePIR addresses this problem for perfect binary trees by exploiting the path structure rather than treating the proof nodes as an arbitrary batch [22]. Its organizing principle is to partition proof nodes into color subdatabases so that a proof path contains at most one node from each color, allowing the client to issue one PIR query per color.

B. Sparse Merkle Proofs and Default Completion

A Sparse Merkle Tree (SMT) is a Merkle tree over a fixed logical address space. A binary SMT of height h has 2^h possible leaf coordinates, even when only a small subset of those coordinates is occupied. A membership proof for an occupied leaf still has the standard verifier interface: one sibling digest per level and hence a height- h proof array. The sparse-tree feature relevant to this work is that many of these proof positions need not be retrieved from the server. If the sibling subtree at a level is empty, its digest is the corresponding public default hash [16].

Thus an SMT proof naturally separates into two classes of entries. *Real sibling digests* are proof material that cannot be reconstructed from public defaults and therefore must be retrieved through PIR. *Default-empty digests* are public constants that the client can reconstruct locally from the default-hash chain. SparseTreePIR preserves the standard SMT verification interface; it changes only the PIR-facing representation of the proof material. In particular, the client still reconstructs a complete height- h proof, while the PIR-facing database stores

only the real sibling digests that may appear in membership proofs.

Figure 2 illustrates the two views of the same SMT snapshot. The verifier reasons about the full logical coordinate tree. The retrieval layer, however, should not materialize empty coordinates as PIR records: non-default nodes carry the only proof material that must be retrieved, while empty proof levels are filled from the public default-hash chain.

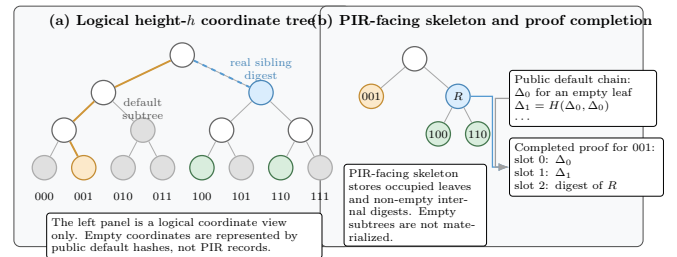


Fig. 2. Logical and storage views of an SMT. The verifier reasons about a complete height- h coordinate tree, but the PIR-facing database should not store every empty coordinate. A membership proof is completed by combining PIR-retrieved real sibling digests with public default hashes.

C. From Proof Privacy to Batch PIR

A straightforward way to hide the requested SMT proof is to run an independent PIR query for every proof level. This approach hides the target, but it treats the height- h proof

array as h unrelated PIR lookups and ignores the structure of the authenticated path. Generic batch-PIR mechanisms can retrieve multiple records privately, but they are designed for arbitrary batches and do not decide which SMT proof objects should become PIR records.

The TreePIR principle suggests a more structured query pattern: partition the relevant proof records so that one proof needs at most one real record from each subdatabase, and send one PIR subquery to every subdatabase. For SMTs, however, the object to be partitioned is not the full logical coordinate tree. Public default positions and inactive coordinates should not become PIR database records. The appropriate PIR-facing object is the set of real sibling-side nodes that can serve as membership-proof material for occupied targets. Once these active proof records are identified, they can be partitioned into color subdatabases under the same fixed-shape retrieval discipline: one subquery per color, real when the color contains a target proof node and dummy otherwise.

D. PIR-Facing Workload Metrics

We treat the underlying PIR scheme as a black-box backend and evaluate the database profile exposed to that backend. For a fixed SMT snapshot, a retrieval layout exposes four quantities to the PIR layer: the number of proof records that must be retrieved through PIR, the number of PIR subqueries sent per membership proof, the largest searched subdatabase, and the size of the public lookup metadata. These parameters do not fully determine the cost of every PIR backend, because practical PIR systems differ in constants, packing strategies, and query/response formats [26], [27], [32], [33], [35], [36]. They do determine the backend-facing workload profile. Sequential execution scales with the queried color stores, while parallel execution is typically limited by the largest searched subdatabase.

This resource view also clarifies the comparison target. Once target privacy is required, the relevant baseline is not ordinary non-private SMT proof serving. The relevant question is how different privacy-preserving layouts expose proof material to PIR: how many PIR records are stored, how many fixed-shape subqueries are sent, how large the searched subdatabases become, and how much public metadata is needed for client-side lookup.

This leaves a concrete retrieval-design problem: for a fixed SMT snapshot, which proof records should be placed in PIR subdatabases, how should these subdatabases be shaped, and how should the client reconstruct the standard height- h SMT proof from private replies and public default hashes? Section V presents SparseTreePIR, our construction for this problem.

III. RELATED WORK

Merkle trees and authenticated data structures. Merkle’s original hash-tree construction [1] underlies a broad family of authenticated data structures. Classical work on authenticated dictionaries and generic authenticated data structures studies how to make dictionary operations verifiable [6], [41]. Deployed systems use Merkle-style commitments in several

roles: tamper-evident and certificate-transparency logs [2]–[4], key-transparency, transparency-dictionary, and verifiable key-directory systems [7]–[10], replicated key-value stores [5], append-only authenticated structures [42], and blockchain or stateless-client state commitments [13]–[15]. These works explain why authenticated maps matter. Our leakage model targets a different layer: the fixed SMT snapshot and its public metadata are not hidden; the goal is to hide which occupied target proof is retrieved from that public snapshot.

Sparse Merkle trees and proof serving. SMTs refine Merkle authentication for large logical address spaces by assigning public default hashes to empty subtrees [16]. This idea appears naturally in revocation/transparency-style maps, authenticated maps, constrained revocation checking, and state-commitment structures such as Jellyfish Merkle Tree [11], [12], [17]. Recent authenticated-map and rollup work further optimizes proof size, multiproofs, or batch update cost [18], [19], [21]. These systems primarily optimize ordinary proof serving: a server traverses the requested coordinate, returns the non-default siblings, and relies on the verifier to reconstruct default-empty levels. SparseTreePIR preserves these proof semantics but changes the retrieval interface. The server should be able to serve the same membership proof without learning which occupied leaf is being checked. Thus the key property for our protocol is not sparsity alone, but *public reconstructibility under private retrieval*: default siblings can remain public, while real sibling digests must be partitioned into PIR subdatabases under a fixed-shape transcript.

Private retrieval of authenticated proofs. Lueks and Goldberg studied private retrieval of authenticated proofs in a multi-client setting [25]; subsequent work explored privacy-preserving Certificate Transparency auditing and proof retrieval with CT-specific mechanisms [23], [24]. The closest layout-level idea is TreePIR [22]: color proof nodes so that one proof can be retrieved with one PIR query per color. We adopt this principle, but change the object being partitioned. TreePIR colors a perfect-tree proof structure; SparseTreePIR colors the active interval forest induced by a fixed SMT snapshot. This is a different retrieval domain, not merely a smaller instance of the same domain: the records are selected by non-default sibling semantics, indexed by served intervals over occupied targets, and completed back into a height- h verifier proof using public default hashes. Thus applying TreePIR to a perfected SMT, or applying generic batch placement to the active records, leaves the main SMT-specific problem unsolved. SparseTreePIR is an SMT proof-retrieval layout, not a competing PIR primitive.

Batch and practical PIR. PIR research began with multi-server information-theoretic constructions [26] and single-server computational PIR [27], [28]. Generic batch retrieval has a long line of work based on batch codes and combinatorial batch codes [29], [30]. The practical PBC line was brought into PIR prominently by Angel, Chen, Laine, and Setty’s 2018 SealPIR work, which uses probabilistic batch codes and Cuckoo-style placement to amortize multi-query PIR processing [31], [33]. More recent batch or multi-query

TABLE I
ORDINARY SMT PROOF-SERVING WORK OPTIMIZES PROOF GENERATION, STORAGE, OR UPDATE COST, WHEREAS SPARSETREEPIR ADDS A TARGET-PRIVATE RETRIEVAL LAYER

Work	Reported goal/result	Optimized layer	Relation to our setting
Efficient SMTs [16]	Formal SMT operations and caching; audit paths for membership/non-membership can be generated in practically constant time, reported below 4 ms with SHA-512/256	Plain proof generation	Non-private lower bound: server knows the queried coordinate
Jellyfish Merkle Tree [17]	SMT variant for Diem state storage; optimizes node keys, node types, proof format, LSM-tree storage, I/O, and computation	Deployed state-tree storage	Representative SMT-system baseline, but proof requests are visible lookups
Compact Merkle multiproofs [18]	Sparse multiproof encoding keeps only k leaf indices instead of an index for every non-leaf hash in a multiproof	Multi-target proof compression	Reduces proof payload for known targets; does not hide which targets are requested
Rollup SMT batch update [19]	For zkSync Lite-style updates, improves account update time from $O(\log n)$ to $O(1)$ and reduces batch update cost by a one-pass traversal	State update/root recomputation	Orthogonal to retrieval privacy; update cost is not the bottleneck studied here
Multicollision-resistant non-membership tree [21]	Reduces authenticated (non-)membership tree depth for ZK-oriented settings while preserving security parameters	Alternative accumulator/proof structure	Changes the authenticated structure; our work keeps standard SMT proof semantics while privatizing retrieval

PIR systems further reduce the cost of retrieving several records through vectorized processing or constant-weight-code techniques [39], [40]. Practical PIR backends have also improved server computation, communication, response size, and packing, including XPIR, OnionPIR, Spiral, SimplePIR, PIANO, and YPIR [32], [34]–[38]. These systems determine cryptographic constants and packing efficiency; our contribution determines the database profiles they query. We do not propose a new cryptographic PIR primitive. Instead, we partition SMT proof records into color subdatabases so that a single proof becomes a fixed-shape batch workload: one subquery per color, with real and dummy targets hidden by the underlying PIR security.

Baselines induced by prior work. Existing SMT systems already optimize the *ordinary* proof-serving path: the server stores a sparse/default-completable authenticated map, traverses the target coordinate, and returns the sibling digests needed by the verifier [11], [12], [16], [17]. Some default-empty siblings can be omitted because the client reconstructs them from the public default-hash chain. This semantic property is the starting point for our construction, but prior SMT proof serving is not a target-private retrieval pattern: the server learns the requested key or path. Table II separates what each prior line provides from what remains missing for fixed-shape batch PIR. It also motivates the evaluation methodology: we compare against prior-work-induced privacy-preserving layouts rather than only against internal ablations.

Balancing PIR-facing color stores. After an active-color layout is chosen, the database layout still matters. A coloring that is valid but highly imbalanced can force one PIR subquery to operate over a much larger color class than the others. Perfect-tree coloring explains why colors reduce the proof-retrieval width, but it does not directly solve the SMT balance problem because active intervals can be highly uneven. Our ActiveBalance algorithm addresses this layout issue on the active interval forest itself. The node-local rules used in the experiments are baselines only; the proposed algorithmic

contribution is the subtree-level load refinement used to flatten the resulting PIR-facing color stores.

IV. PROBLEM FORMULATION AND LEAKAGE MODEL

This section defines the retrieval object studied in this paper. We work with one fixed public SMT snapshot and hide the queried occupied leaf within that snapshot. The snapshot’s occupancy pattern is public, and the formal protocol targets membership-proof retrieval. Non-membership queries over arbitrary absent logical coordinates require served intervals over the full logical coordinate space rather than over occupied-leaf ranks, so that extension is outside the present scope.

We use *private record* only as shorthand for a record that must be retrieved through PIR rather than reconstructed from public default hashes; it is not secret from the storage server. The privacy goal is query privacy: the server should not learn which record, authenticated path, or target leaf is selected beyond the setup leakage defined below.

A. Fixed-Snapshot Active Proof Objects

Let T be a fixed binary Sparse Merkle Tree of height h , and let $\mathcal{L}$ denote its set of real leaves. Hashes are still computed in the standard bottom-up way. The active-interval view is used only for retrieval layout and query modeling; it does not alter the hash-construction process. We assume the standard public default-empty hash chain

$$\Delta_0, \Delta_1, \dots, \Delta_{h-1},$$

where Δ_0 is the default empty-leaf hash and $\Delta_{t+1} = H(\Delta_t \| \Delta_t)$ for $0 \leq t < h - 1$. Thus, every empty sibling subtree of height t has publicly reconstructible digest Δ_t .

Definition IV.1 (Active proof-bearing node). *For a node u in T , let $\text{cnt}(u)$ be the number of real leaves in the subtree rooted at u . Let u be a non-root node, and let $\text{sib}(u)$ be its sibling. We say that u is an active proof-bearing node if*

$$\text{cnt}(u) > 0 \quad \text{and} \quad \text{cnt}(\text{sib}(u)) > 0.$$

TABLE II
PRIOR-WORK-INDUCED LAYOUTS FOR SMT PROOF RETRIEVAL

Prior line	Natural proof object	Induced baseline	Missing for our goal
Standard SMT proof APIs [11], [16], [17]	Height- h proof path with real siblings and public defaults	Plain SMT proof serving	No target privacy; server sees the queried coordinate/path
Compact SMT storage [12], [16], [17]	Non-default skeleton plus public default chain	Active-only proof serving	Saves storage, but gives no fixed-shape PIR query pattern
Generic batch-PIR placement [29], [30], [33]	Active proof records treated as an arbitrary batch	PBC-style SMT; flat active PIR	Ignores ancestor/path structure; may query or replicate too much
TreePIR on perfect trees [22]	Colored proof nodes in a perfect-tree layout	Perfectized and pruned TreePIR	Colors the full coordinate tree, including public default positions
This paper	Active sibling digests indexed by served intervals	SparseTreePIR	Active-width batch PIR with local default completion

The set of all active proof-bearing nodes is denoted by $\mathcal{A}(T)$.

The first condition ensures that u itself carries a real digest rather than a public default-empty digest. The second condition ensures that u is the sibling-side node for at least one real target leaf. Thus $\mathcal{A}(T)$ contains exactly the non-empty sibling digests that may have to be retrieved through PIR. Empty sibling subtrees remain outside the PIR database because their digests are publicly reconstructible from the default hash chain.

Proposition IV.1 (Minimal non-default PIR record set). *Fix a public height- h SMT snapshot T and the public default-empty hash chain, and assume the SMT hash function is collision resistant. In the record-based retrieval model where a PIR record supplies a proof sibling digest and public defaults may be filled locally, $\mathcal{A}(T)$ is exactly the set of non-default sibling digests that may be required by some occupied-leaf membership proof, except with negligible hash-collision probability. Consequently, any PIR-facing database that supports all occupied-leaf membership proofs must make one digest for every node in $\mathcal{A}(T)$ retrievable, while no default-empty sibling position needs a PIR record.*

Proof. If $u \in \mathcal{A}(T)$, then u is non-empty and $\text{sib}(u)$ is also non-empty. Choose any occupied leaf in the subtree of $\text{sib}(u)$; the verifier-facing membership proof for that leaf contains the digest of u as a sibling digest. By collision resistance, a non-empty subtree digest is computationally distinct from the corresponding public default-empty digest except with negligible probability. Thus a record-based private retrieval layout that reconstructs every occupied-leaf proof must make this digest retrievable. Conversely, if a proof sibling subtree is empty, its digest is the public value Δ_t for the corresponding height t and is reconstructed locally. A non-empty node whose sibling subtree is empty is not a sibling for any occupied target leaf. Therefore the necessary and sufficient non-default PIR record set is precisely $\mathcal{A}(T)$.

Definition IV.2 (Active proof path). *For any real leaf $\ell \in \mathcal{L}$, let*

$$\text{Path}(\ell) \subseteq \mathcal{A}(T)$$

denote the sequence of all real, PIR-retrievable nodes in the complete Merkle proof of ℓ , ordered from higher to lower proof levels. We call $\text{Path}(\ell)$ the active proof path of ℓ .

Equivalently, $\text{Path}(\ell)$ contains exactly those sibling nodes on the standard height- h proof of ℓ whose subtrees contain at least one real leaf. Sibling levels whose subtrees are empty are not active proof nodes and are later filled locally with the corresponding default hash.

Although the full SMT proof length is always h , the number of real nodes on $\text{Path}(\ell)$ may be smaller than h , because some levels correspond only to publicly reconstructible default empty nodes.

Let

$$m = \max_{\ell \in \mathcal{L}} |\text{Path}(\ell)|$$

be the *active width* of the snapshot, namely the maximum number of non-default proof digests that can appear in one occupied-leaf membership proof. By construction, $m \leq h$. Once the active-node retrieval model is fixed, this same quantity is also the active color count and the active batch width. Throughout the paper, $[m]$ denotes $\{1, \dots, m\}$.

The case $|\mathcal{L}| = 1$ is degenerate: no occupied target has a non-default sibling subtree, so $\mathcal{A}(T) = \emptyset$, $m = 0$, and no PIR query is issued. Unless explicitly stated otherwise, the rest of the paper assumes $|\mathcal{L}| \geq 2$.

The role of m is analogous to the role of h in TreePIR only after public default-empty levels have been removed from the PIR-facing layout. It is not a tunable parameter chosen by the balancing algorithm; it is a public resource parameter of the fixed SMT snapshot. The full verifier still receives a height- h proof after completion. The active width only measures how many proof positions may require private retrieval under the active-node retrieval model.

Remark IV.1 (Why m can be much smaller than h). *The value of m is necessarily instance-dependent because it depends on where the occupied leaves branch. The setup phase publishes m as part of the public layout, just as a perfect-tree TreePIR setup publishes height h . To interpret this parameter, suppose n occupied leaves are sampled uniformly from a logical universe of size $U = 2^h$, and condition on one target leaf. At proof level t above the leaf, the sibling subtree contains 2^t*

logical leaves, so the probability that this level contributes an active private sibling is

$$1 - \frac{\binom{U-1-2^t}{n-1}}{\binom{U-1}{n-1}} \approx 1 - \exp\left(-\frac{(n-1)2^t}{U-1}\right).$$

Thus the expected active proof length of a random target is the sum of these probabilities over $t = 0, \dots, h-1$. When $n \ll U$, most low levels have empty sibling subtrees and only the levels near the branching depth of the occupied set contribute; the active width therefore tracks the occupied-leaf skeleton rather than the logical height. In adversarial layouts, the sharp worst-case bound remains $m \leq \min\{h, n-1\}$.

This paper focuses on target-private retrieval of *membership* proofs for occupied leaves in a fixed public SMT snapshot. This scope matches the proof-retrieval setting in which the client privately asks for the authentication material of an existing committed object. Non-membership retrieval is a natural extension, but it changes the object being hidden. The target domain becomes the full logical coordinate space rather than the public occupied-leaf set; a proof may certify an empty coordinate, a divergence from the nearest occupied prefix, or a gap between existing keys; and the protocol must specify whether the server should learn that the query is for an absent coordinate. Consequently, the served intervals would have to range over logical coordinates or absent-coordinate regions, not only over occupied-leaf ranks. We therefore keep non-membership SparseTreePIR outside the present formal model and treat it as a separate extension requiring its own metadata and leakage model.

B. Served Intervals

To support efficient indexing, we associate each active node with the interval of target leaves for which it acts as a proof sibling. This is not the subtree interval of u itself. Order the real leaves by their logical SMT positions and let $r(\ell) \in \{0, \dots, |\mathcal{L}| - 1\}$ be the real-leaf rank of ℓ . Let $s = \text{sib}(u)$, and define the *served interval*

$$I_{\text{srv}}(u) = [L(s), R(s)]$$

as the interval of real-leaf ranks covered by the sibling subtree of u . Thus $L(s)$ and $R(s)$ are rank endpoints, not heap-index endpoints in the full logical coordinate space. Then, for any real leaf ℓ ,

$$u \in \text{Path}(\ell) \iff r(\ell) \in I_{\text{srv}}(u).$$

Since every binary subtree corresponds to a contiguous interval in leaf order, the resulting served intervals form a laminar family: any two intervals are either disjoint, or one contains the other. This structure underlies both our coloring method and our indexing method.

C. Active Coloring and Load Metrics

The constructions below use m active colors, where m is the active width defined above. Since $m \leq h$, the resulting active batch profile is no wider than the SMT verifier height.

Section VI later states the corresponding model-specific lower-bound argument: under the active-node, one-query-per-color model, a proof path containing m active nodes requires m distinct colors.

Definition IV.3 (Valid ancestral coloring). *A map*

$$\varphi : \mathcal{A}(T) \rightarrow [m]$$

is called a valid ancestral coloring if for every real leaf $\ell \in \mathcal{L}$ and every two distinct nodes $u, v \in \text{Path}(\ell)$, we have

$$u \neq v \implies \varphi(u) \neq \varphi(v).$$

A valid ancestral coloring ensures that along every real proof path, colors are pairwise distinct. Hence, in each color subdatabase the client needs to retrieve at most one real node.

For a color $c \in [m]$, let

$$\mathcal{D}_c = \{u \in \mathcal{A}(T) : \varphi(u) = c\}$$

be the corresponding subdatabase, with size $N_c = |\mathcal{D}_c|$. Let $N = |\mathcal{A}(T)|$, and let

$$\bar{N} = N/m$$

be the average subdatabase size. The main optimization target in this paper is to keep the color subdatabases as balanced as possible while preserving valid active coloring. In particular, we track

$$M(\varphi) = \max_c N_c, \quad \Delta_N(\varphi) = \max_c N_c - \min_c N_c,$$

and the normalized largest-bucket ratio

$$R(\varphi) = \frac{M(\varphi)}{\bar{N}}.$$

Here $M(\varphi)$ is the direct structural bottleneck, while $\Delta_N(\varphi)$ and $R(\varphi)$ indicate how far the partition is from an equal split.

The ActiveBalance scheme is self-contained. It first constructs a feasible active coloring by a first-fit traversal of the interval forest, and then refines that coloring by subtree-level color permutations that directly improve the global bucket loads.

Definition IV.4 (Client public inputs). *For a target leaf ℓ , the client is assumed to know the following public auxiliary inputs before issuing PIR queries:*

- 1) the tree height h and the position of ℓ in the SMT;
- 2) the default-empty hash chain $\Delta_0, \dots, \Delta_{h-1}$;
- 3) the real-leaf rank $r(\ell)$ of the target within the public ordered set of occupied leaves;
- 4) for each active color $c \in [m]$, a public metadata table

$$\mathcal{M}_c = ((I_{\text{srv}}(u), \delta(u)))_{u \in \mathcal{D}_c},$$

sorted by interval, where the two endpoints of $I_{\text{srv}}(u)$ are real-leaf ranks and $\delta(u) = h - \text{depth}(u)$ denotes the proof level of u measured from leaf to root.

The PIR subdatabases store only retrieved node digests. The level and interval information required for local proof reconstruction is carried by the public metadata tables rather than by PIR records.

TABLE III
SERVER-VISIBLE SETUP STATE AND HIDDEN QUERY INFORMATION

Visible to the storage server	Hidden by PIR queries
root, occupied-leaf set, default chain, active-node set, color stores, interval metadata, color assignment, color-store sizes, active color count m	target leaf ℓ , active proof path, which color slots are real or dummy, selected item index inside each color store

D. Setup Leakage and Target Privacy

The retrieval protocol in this paper hides the queried target leaf inside one *fixed public SMT instance*; it does not aim to hide the entire occupancy pattern of that instance. We model the storage server’s snapshot-level view with the following deterministic setup leakage.

Definition IV.5 (Setup leakage). *Let setup on a fixed SMT snapshot T produce active color count m , active-node set $\mathcal{A}(T)$, coloring $\varphi : \mathcal{A}(T) \rightarrow [m]$, color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, and public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$. Let $N_c = |\mathcal{D}_c|$. The setup leakage visible to the storage server is*

$$\mathcal{L}_{\text{setup}}(T) = \left(h, \text{root}(T), \mathcal{L}, \{\Delta_t\}_{t=0}^{h-1}, \mathcal{A}(T), m, \varphi, \{(\mathcal{D}_c, \mathcal{M}_c, N_c)\}_{c=1}^m \right).$$

Thus the leakage includes the fixed occupancy pattern, the committed root, the public default chain, the complete active layout, and the server-side database layout. For an online target ℓ , it does not include ℓ , the active proof path $\text{Path}(\ell)$, the set of real-query colors, or the selected item index inside any color subdatabase. The only target-independent online shape revealed by the protocol is that every retrieval sends exactly one PIR subquery to each of the m active color stores.

The hidden query information consists of the target leaf ℓ , equivalently the active proof path $\text{Path}(\ell)$, as well as which color positions correspond to real retrievals rather than dummy subqueries and which item index is privately selected inside each color subdatabase.

We make this leakage profile explicit. The retrieval layout is precomputed once for a fixed SMT snapshot and is treated as server-visible structure thereafter. Our privacy claim is therefore *query privacy relative to a server-visible active proof-bearing layout*, rather than structure privacy for the whole sparse tree. The corresponding transcript view is fixed-shape: the server sees one well-formed PIR query per color, while the real/dummy roles are client-local.

Definition IV.6 (Target privacy game). *Let $\mathcal{L}_{\text{setup}}(T)$ be the setup leakage in Definition IV.5. In the experiment $\text{PrivTarget}_{\mathcal{A}}(\lambda)$, the challenger runs setup for the fixed SMT snapshot and gives $\mathcal{L}_{\text{setup}}(T)$ to the adversary $\mathcal{A}$. The adversary chooses two real target leaves $\ell_0, \ell_1 \in \mathcal{L}$. The challenger samples $b \leftarrow \{0, 1\}$, runs the batch query-generation algorithm for ℓ_b , and gives the full batch query transcript to $\mathcal{A}$.*

The adversary outputs a bit b' . Its advantage is

$$\text{Adv}_{\mathcal{A}}^{\text{target}} = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

The protocol is *target-private relative to $\mathcal{L}_{\text{setup}}(T)$* if this advantage is negligible for every efficient adversary.

For dummy positions, the client samples a well-formed PIR query to a valid index in the same color subdatabase, independently of the target leaf, and discards the response. The setup layout exposes only active, non-empty color subdatabases. Implementations that allow empty color classes can remove them during setup or pad them with one public dummy record.

V. SPARSETREEPIR CONSTRUCTION

This section turns the model into a concrete retrieval construction. The offline algorithm builds the PIR-facing databases from the compressed non-default skeleton, not from a materialized full tree. The online algorithms then show how the client generates a fixed-shape batch query and locally completes the standard height- h SMT proof.

A. Construction Overview

SparseTreePIR instantiates a snapshot-specific retrieval layout with a fixed online transcript. During setup, the server extracts active proof-bearing nodes, partitions them into color subdatabases, and publishes metadata that maps target ranks to served intervals. During online retrieval, the client sends exactly one PIR subquery to each color subdatabase. If the target proof contains a real node in that color, the query selects that node; otherwise, the query is a dummy sampled through the same PIR interface. After receiving the responses, the client writes the real digests into the appropriate proof slots and fills the remaining slots with public default hashes. Figure 3 summarizes this setup-and-retrieval flow.

Online retrieval pattern. Given a fixed public SMT snapshot T , setup outputs active color subdatabases $\mathcal{D}_1, \dots, \mathcal{D}_m$, public metadata tables $\mathcal{M}_1, \dots, \mathcal{M}_m$, and m active color classes. For every occupied target leaf, the online retrieval pattern sends exactly one PIR subquery to each $\mathcal{D}_c$. A subquery is real if color c contains the active sibling digest serving the target leaf, and dummy otherwise. The client then combines the real PIR outputs with the public default-empty hash chain to reconstruct the standard height- h SMT proof. The backend-facing resources of this pattern are PIR storage $|\mathcal{A}(T)|$, active batch width m , largest color store $\max_c |\mathcal{D}_c|$, and public metadata size $O(|\mathcal{A}(T)|)$.

The algorithms below are presented as a protocol stack rather than as independent routines. Algorithm 1 is the server-side setup procedure for one fixed public SMT snapshot. It extracts the active proof-bearing nodes, computes the active-width parameter, invokes Algorithm 2 as a balancing subroutine, and publishes the metadata consumed by the client. Algorithm 3 is the online client lookup and query-generation procedure. It consumes exactly the public outputs of setup and produces two outputs: the batch of PIR subqueries sent to the

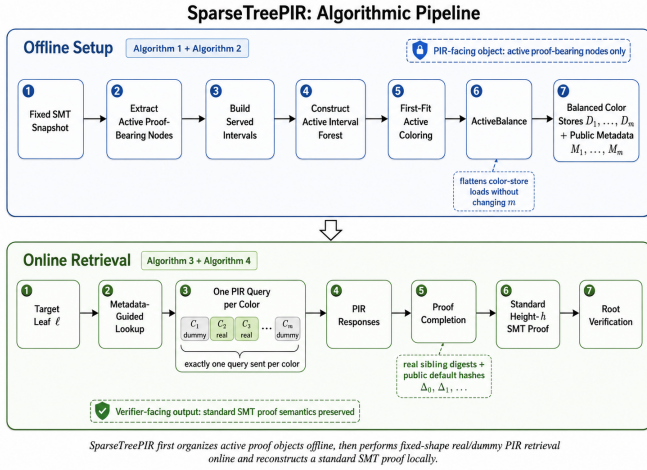


Fig. 3. SparseTreePIR algorithmic pipeline. Offline setup extracts active proof-bearing nodes, builds served intervals, colors the active interval forest, and balances color stores. Online retrieval uses public metadata to issue exactly one PIR query per color, then combines retrieved real sibling digests with public default hashes to reconstruct the standard height- h SMT proof.

server, and a local target map that is never revealed. After the underlying PIR backend returns one response per color, Algorithm 4 uses that local map, the public metadata, and the default-empty hash chain to expand the real replies into the standard height- h SMT proof.

This separation mirrors the presentation style of TreePIR: setup fixes the colored database layout once, and every online retrieval has the same external profile. The server observes one PIR query per color store. Only the client knows which of those positions are real proof nodes and which are dummy placeholders.

B. Running Example

Figure 4 presents the running example used in this section. It shows a complete height-4 coordinate tree for explanation, but the implementation traverses the compressed non-default skeleton and stores only the active proof-bearing nodes in color subdatabases. Because the model has already defined active nodes, served intervals, and valid coloring, the example can now be interpreted as an instance of the formal interface rather than as an additional rule.

The example separates three layers that are otherwise easy to conflate. The first is the *full SMT coordinate tree*, which still has the usual SMT geometry and from which all hashes are derived. The second is the *active proof-bearing structure*: only sibling-side nodes that can appear in at least one real proof are stored in PIR subdatabases. The third is the *public completion layer*: every untouched proof level is reconstructed from the default-empty hash chain. Thus the scheme keeps the verifier’s coordinate system intact, privatizes only the active proof-bearing layer, and leaves default completion public. The figure writes served sets using logical leaf labels for readability; the actual public metadata stores real-leaf rank intervals.

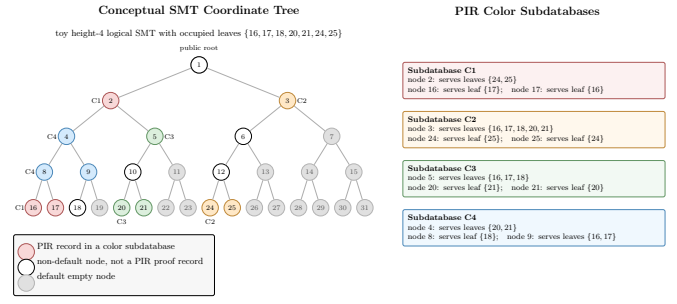


Fig. 4. Running SMT example and its final color subdatabases. The full coordinate tree is explanatory; only active proof-bearing nodes are stored in PIR subdatabases.

The occupied leaves are $\{16, 17, 18, 20, 21, 24, 25\}$. The active proof-bearing nodes are exactly the twelve nodes shown in the color subdatabases of Figure 4:

$$\{2, 3, 4, 5, 8, 9, 16, 17, 20, 21, 24, 25\}.$$

The longest active proof path has length 4, so the setup exposes four active color classes in this example.

This instance is useful because it already exhibits the balance problem. The first-fit feasibility stage may produce the legal color split

$$\begin{aligned} \mathcal{D}_1 &= \{3, 2\}, & \mathcal{D}_2 &= \{5, 4, 25, 24\}, \\ \mathcal{D}_3 &= \{9, 8, 21, 20\}, & \mathcal{D}_4 &= \{17, 16\}, \end{aligned}$$

whose bucket sizes are $(2, 4, 4, 2)$. The coloring is valid but imbalanced. By contrast, the subtree-level refinement can transform the same active forest into

$$\begin{aligned} \mathcal{D}_1 &= \{2, 17, 16\}, & \mathcal{D}_2 &= \{3, 25, 24\}, \\ \mathcal{D}_3 &= \{5, 21, 20\}, & \mathcal{D}_4 &= \{4, 9, 8\}, \end{aligned}$$

which is still valid and now has perfectly balanced bucket sizes $(3, 3, 3, 3)$. This final split is deliberately not a layer coloring: for example, nodes 4, 8, and 9 share color 4 even though they lie at different logical depths in the full coordinate tree, while depth-1 nodes 2 and 3 have different colors. This is possible because the constraint is induced by served proof intervals, not by raw tree depth. Figure 5 visualizes this change. In the current algorithm, this improvement is reached by a short sequence of accepted legal subtree permutations; the active query pattern stays fixed throughout. We will reuse the same small instance again when illustrating one concrete query and local proof completion.

C. Offline Active-Interval Setup

The server-side preprocessing stage takes one fixed SMT instance and produces the data needed for online private retrieval. It does not materialize a perfected proof-bearing tree. Instead, it traverses the compressed non-default skeleton induced by the occupied leaves, extracts the active proof-bearing nodes, builds the laminar interval forest, computes the active-width parameter m , and then colors the nodes under that active-node retrieval model.

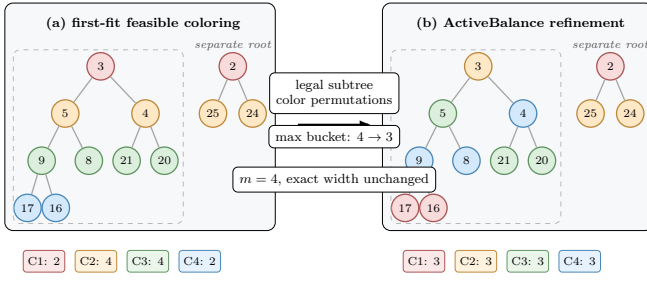


Fig. 5. Running example for the coloring stage. The first-fit feasibility stage leaves an uneven but valid split, while ActiveBalance applies accepted legal subtree permutations and reaches a flatter partition on the same active forest.

Algorithm 1 Setup: active-interval layout

Input: Fixed public SMT snapshot T , occupied leaf set $\mathcal{L}$, balancing rounds R_1
Output: Active color count m , active color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, valid coloring φ

- 1: Sort the occupied leaves by logical SMT position and assign real-leaf ranks $0, \dots, |\mathcal{L}| - 1$
- 2: Build the compressed non-default skeleton $\mathcal{S}(T)$ induced by the occupied leaves
- 3: Annotate each skeleton edge with its corresponding original SMT child root and skipped logical levels
- 4: Initialize $\mathcal{A}(T) \leftarrow \emptyset$
- 5: **for** each branching skeleton node p with two non-default original children **do**
- 6: **for** each original child root u of p **do**
- 7: $s \leftarrow \text{sib}(u)$ in the full SMT coordinate system
- 8: Add u to $\mathcal{A}(T)$
- 9: Assign served rank interval $I_{\text{srv}}(u) = [L(s), R(s)]$ and proof level $\delta(u)$
- 10: **end for**
- 11: **end for**
- 12: Sort all active intervals by increasing left endpoint and decreasing interval length
- 13: Build the interval forest $\mathcal{F}$ from interval-containment relations
- 14: Set m to the maximum depth of $\mathcal{F}$
- 15: Compute $\varphi \leftarrow \text{FIRSTFITCOLORING}(\mathcal{F}, m)$
- 16: Refine $\varphi \leftarrow \text{ActiveBalance}(\mathcal{F}, \varphi, R_1)$ using Algorithm 2
- 17: Build the private subdatabases $\mathcal{D}_c$ and the public metadata tables $\mathcal{M}_c$
- 18: **return** $m, \{\mathcal{D}_c\}_{c=1}^m, \{\mathcal{M}_c\}_{c=1}^m, \varphi$

The first-fit stage is only a feasibility step. It traverses each tree of $\mathcal{F}$ from root to leaves and assigns every node the smallest color in $[m]$ not used on its strict ancestor chain. Since no active proof path contains more than m active nodes, such a color always exists. The actual balancing contribution is the subsequent ActiveBalance subtree refinement. The protocol-complete formulation remains specific to fixed binary SMTs, because public proof completion relies on the standard default-empty hash chain.

From a systems viewpoint, Algorithm 1 is the offline setup stage of the construction. Once the color subdatabases and public metadata tables have been built for one fixed SMT instance, the online retrieval protocol never needs to see the whole tree again. The expensive structural work is pushed offline, while the online stage sees only interval tables, color subdatabases, and the public default-hash chain.

Algorithm 2 ActiveBalance subtree refinement

Input: Interval forest $\mathcal{F}$, valid active coloring φ , rounds R_1

Output: Updated valid active coloring φ

- 1: Compute global loads $\{N_c\}_{c=1}^m$ and $\Psi^* \leftarrow \Psi(\varphi)$
- 2: **for** $t = 1$ **to** R_1 **do**
- 3: Annotate each subtree root U with ancestor colors and histogram $\{C_U(c)\}_{c=1}^m$
- 4: $\text{best} \leftarrow \text{NONE}$
- 5: **for** each subtree root U in $\mathcal{F}$ **do**
- 6: $A(U) \leftarrow [m] \setminus \text{AncestorColors}(U)$
- 7: **if** $|A(U)| \leq 1$ **then**
- 8: **continue**
- 9: **end if**
- 10: Build $K_U[a, b] = (N_b - C_U(b) + C_U(a))^2$ for $a, b \in A(U)$
- 11: Let π_U be the min-cost perfect matching on K_U
- 12: Form φ' by applying π_U inside U and leaving all nodes outside U unchanged
- 13: **if** $\Psi(\varphi') < \Psi^*$ **then**
- 14: $(\text{best}, \Psi^*) \leftarrow ((U, \pi_U, \varphi'), \Psi(\varphi'))$
- 15: **end if**
- 16: **end for**
- 17: **if** $\text{best} = \text{NONE}$ **then**
- 18: **break**
- 19: **end if**
- 20: Apply best and update $\varphi, \{N_c\}_{c=1}^m$, and Ψ^*
- 21: **end for**
- 22: **return** φ

D. ActiveBalance: Subtree-Level Load Refinement

The balancing component is the subtree-level refinement. Instead of moving one node at a time, ActiveBalance treats a rooted subtree as a whole color-load vector and tries to permute its internal colors so that the global color loads become flatter. We compare candidate refinements by the lexicographic signature

$$\Psi(\varphi) = \text{sort}_{\downarrow}(N_1, \dots, N_m),$$

so the objective is to reduce the largest subdatabase first and then flatten the remaining count vector.

For a rooted subtree U of the interval forest and a color $c \in [m]$, let $C_U(c)$ denote the number of nodes of color c currently appearing inside U . Let $A(U) \subseteq [m]$ be the set of colors not used on the strict ancestor chain of U . Since every node in U must already be colored from $A(U)$, a legal refinement is any permutation π_U of $A(U)$ applied uniformly to the colors inside U . Proposition VI.1 shows that every such permutation preserves validity.

Algorithm 2 makes two design choices explicit. First, each subtree is evaluated against a count-only quadratic assignment objective rather than against the full combinatorial search over all possible recolorings. The cost term $(N_b - C_U(b) + C_U(a))^2$ is the squared size of bucket b after the portion of color b currently inside U is removed and the portion of local color a is mapped into it. Thus the surrogate favors moving heavier local color mass toward lighter global buckets. Second, after solving that local assignment, we accept a move only if the true global signature $\Psi(\varphi)$ decreases. Hence the surrogate guides the search, while the lexicographic bucket vector remains the actual acceptance rule.

E. Fixed-Shape Online Retrieval

After offline setup, the client needs only public interval metadata to determine which color positions contain real proof material for the target leaf. Since same-color intervals are disjoint, one binary search per color is sufficient.

Algorithm 3 Online query generation: one PIR subquery per color

Input: Target leaf ℓ with real-leaf rank $r(\ell)$, color subdatabases $\{\mathcal{D}_c\}_{c=1}^m$, public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, underlying PIR query generator PIR.Query

Output: Batch query $\mathcal{Q} = \{Q_c\}_{c=1}^m$ and private local target map $\{\tau_c\}_{c=1}^m$

- 1: Initialize $\mathcal{Q} \leftarrow \emptyset$
- 2: **for** $c = 1$ **to** m **do**
- 3: Perform a binary search for $r(\ell)$ in the served-rank interval projection of $\mathcal{M}_c$
- 4: **if** there exists exactly one interval containing $r(\ell)$ **then**
- 5: Let j_c be the position of that node in $\mathcal{D}_c$
- 6: Set $\tau_c \leftarrow j_c$ and $Q_c \leftarrow \text{PIR.Query}(\mathcal{D}_c, j_c)$
- 7: **else**
- 8: Set $\tau_c \leftarrow \perp$ and sample $d_c \leftarrow \text{DummyIndex}(\mathcal{D}_c)$
- 9: Set $Q_c \leftarrow \text{PIR.Query}(\mathcal{D}_c, d_c)$
- 10: **end if**
- 11: Add Q_c to $\mathcal{Q}$
- 12: **end for**
- 13: Send $\mathcal{Q}$ to the server and keep $\{\tau_c\}$ locally
- 14: **return** $\mathcal{Q}, \{\tau_c\}_{c=1}^m$

F. Local Proof Completion

The online retrieval phase ends with a purely local completion step. Real PIR outputs are written into the proof levels indicated by the public metadata, while all untouched levels are filled by the default-empty hash chain. No additional server round is required after batch extraction.

Figure 6 illustrates how dummy responses coexist with proof completion. The client privately retrieves only the levels whose sibling subtrees contain real data, fills all other slots from the public default chain, and then performs the standard bottom-up SMT verification. In the example, leaf 18 retrieves nodes 3, 5, and 8, sends a dummy query for color 1, and keeps the remaining proof slot equal to Δ_0 .

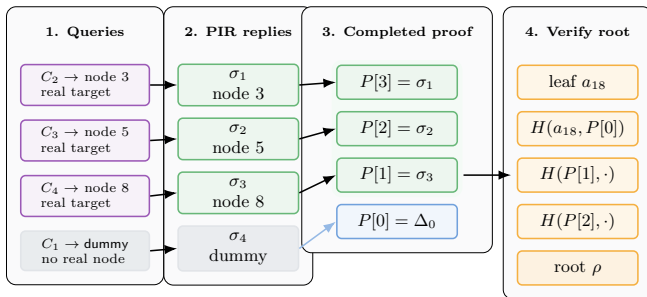


Fig. 6. One fixed-shape batch query and local proof completion for leaf 18.

Algorithm 4 Client-side proof expansion and root reconstruction

Input: Target leaf coordinate ℓ , target leaf digest a_0 , public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$, private local target map $\{\tau_c\}_{c=1}^m$ from Algorithm 3, extracted PIR outputs $\{\sigma_c\}_{c=1}^m$, default-empty hash chain $\{\Delta_t\}_{t=0}^{h-1}$

Output: Complete proof array $P[0..h-1]$ and reconstructed root digest ρ

- 1: **for** $t = 0$ **to** $h - 1$ **do**
- 2: Set $P[t] \leftarrow \Delta_t$
- 3: **end for**
- 4: **for** $c = 1$ **to** m **do**
- 5: **if** $\tau_c \neq \perp$ **then**
- 6: Read $(I_{\text{srv}}, \delta) \leftarrow \mathcal{M}_c[\tau_c]$
- 7: Set $P[\delta] \leftarrow \sigma_c$
- 8: **end if**
- 9: **end for**
- 10: $\rho \leftarrow a_0$
- 11: **for** $t = 0$ **to** $h - 1$ **do**
- 12: **if** the t -th path bit of ℓ is 0 **then**
- 13: $\rho \leftarrow H(\rho \| P[t])$
- 14: **else**
- 15: $\rho \leftarrow H(P[t] \| \rho)$
- 16: **end if**
- 17: **end for**
- 18: **return** P, ρ

G. End-to-End Retrieval Walkthrough

Putting these pieces together, the online flow for one target leaf is as follows. The client uses the public interval tables to identify at most one real target per color, sends one PIR subquery for each color, interprets the extracted outputs through the local target map $\{\tau_c\}$, and writes only the real ones into the corresponding proof levels. It then combines those real responses with the default-empty chain and hashes upward to the root. At the protocol level, every retrieval has the same batch profile of exactly m color positions; sparsity changes only which positions are real rather than dummy, and that distinction is hidden by the PIR layer.

VI. CORRECTNESS, PRIVACY, AND COMPLEXITY

The following claims are stated relative to the active-node, one-query-per-color model just defined. In particular, the active-width result is not a lower bound for every conceivable private proof-retrieval protocol. It characterizes the color count and batch profile for the model where each active proof-bearing node is assigned to one color subdatabase and each retrieval sends one subquery per active color.

Theorem VI.1 (Exact active width for one-query-per-color retrieval). *Let $n = |\mathcal{L}| \geq 2$ and let*

$$m = \max_{\ell \in \mathcal{L}} |\text{Path}(\ell)|.$$

For a fixed height- h SMT snapshot, m is the exact active width of the active-node one-query-per-color model: fewer than m colors cannot support all target proofs, while m colors are

sufficient for a valid ancestral coloring of $\mathcal{A}(T)$. A feasible m -coloring can be computed in $O(N)$ time once the interval forest is available. Moreover,

$$m \leq \min\{h, n - 1\},$$

and this bound is tight for every h and $n \geq 2$.

Proof. For the lower bound, choose a real leaf ℓ^* whose active proof path has length m . All nodes on $\text{Path}(\ell^*)$ appear together in one proof, so any valid ancestral coloring must assign pairwise distinct colors to these m nodes. Hence at least m colors are necessary.

For the upper bound, recall that the served intervals $\{I_{\text{srv}}(u)\}$ form a laminar family, and two active nodes can co-occur in a proof exactly when their served intervals intersect. In a laminar family, interval intersection is equivalent to ancestor-descendant relation in the interval forest. Therefore, assigning every node the color equal to its depth in that forest is valid: two nodes with the same depth cannot be in an ancestor-descendant relation, hence their served intervals are disjoint and they never appear in the same proof. This depth coloring uses exactly the maximum forest depth, which is precisely m . The coloring is computed by a single depth-first traversal, hence in $O(N)$ time.

For the structural bound, every active proof node on the proof path of a target leaf ℓ corresponds to one ancestor where the sibling subtree is non-empty. These ancestors are at distinct SMT levels, so there are at most h of them. Moreover, the non-empty sibling subtrees witnessed along one target path are pairwise disjoint, and each contains at least one occupied leaf different from ℓ . Therefore, one target leaf can have at most $n - 1$ active sibling nodes, proving $m \leq \min\{h, n - 1\}$.

The same observation gives the compressed-skeleton interpretation: after contracting unary non-default paths, only branching ancestors contribute active proof material, and m is the maximum number of such branching nodes on any compressed root-to-leaf path.

For tightness, let $k = \min\{h, n - 1\}$ and choose the target leaf with logical bit string 0^h . For each depth $d = 0, \dots, k - 1$, add one occupied leaf in the sibling subtree of the target path at that depth, for example the bit string $0^d 1 0^{h-d-1}$. These k leaves make all k corresponding sibling subtrees non-empty, so the target has k active proof nodes. If more than $k + 1$ occupied leaves are required, add them arbitrarily without removing any of these active witnesses. Hence the bound is attainable.

Remark VI.1. *Theorem VI.1 is where the SMT formulation departs most clearly from a naive perfected view. In a perfected layout, the batch profile is pinned to the full tree height h regardless of how much real proof material exists. In the direct SMT formulation, the active batch profile is governed instead by the maximum number of active proof positions on any real path. Consequently, the storage gain and the width gain are related but not identical: reducing the stored proof-bearing structure is immediate, while reducing the number of active color queries requires the active proof paths themselves to become shorter.*

Lemma VI.1 (Active-record bound). *Let $n = |\mathcal{L}|$. Then*

$$|\mathcal{A}(T)| \leq 2(n - 1).$$

The bound is tight for every binary occupied-leaf skeleton in which every branching internal node has two non-empty children represented in the SMT.

Proof. Every active proof-bearing node u has a parent p whose two original SMT children are both non-empty: u is non-empty by definition, and $\text{sib}(u)$ is also non-empty. Conversely, each branching parent with two non-empty children contributes at most its two child roots to $\mathcal{A}(T)$. Thus $|\mathcal{A}(T)|$ is at most twice the number of branching internal nodes in the compressed non-default skeleton induced by the occupied leaves.

Any finite rooted binary tree with n leaves has at most $n - 1$ branching internal nodes. Therefore $|\mathcal{A}(T)| \leq 2(n - 1)$. When the occupied-leaf skeleton is a full binary skeleton with $n - 1$ branching internal nodes and both children of each branching node correspond to non-empty SMT subtrees, both child roots are active, giving equality.

Proposition VI.1 (Validity of subtree color permutations). *Let φ be a valid ancestral coloring, let U be any rooted subtree of the interval forest, and let $A(U) \subseteq [m]$ be the set of colors not used on the strict ancestor chain of U . Assume every node in U is colored from $A(U)$. If π is any permutation of $A(U)$, and every node $x \in U$ is recolored from $\varphi(x)$ to $\pi(\varphi(x))$ while all nodes outside U keep their colors, then the resulting coloring is still valid.*

Proof. Consider any two nodes that may appear together on one active proof path. If both lie outside U , nothing changes. If both lie inside U , they were originally assigned distinct colors under φ , and applying the same permutation π preserves distinctness because π is injective on $A(U)$. It remains to consider a pair consisting of one node $x \in U$ and one node $y \notin U$.

If x and y can co-occur on a proof path, then y must be an ancestor of the subtree root of U , because nodes outside U but incomparable with U correspond to disjoint intervals and cannot appear on the same path. By definition, every ancestor color is excluded from $A(U)$. Therefore $\pi(\varphi(x)) \in A(U)$ can never equal $\varphi(y)$, and the pair remains properly colored. Hence no violating ancestor-descendant pair is created, so the recolored assignment is valid.

Proposition VI.2 (Per-color uniqueness on proof paths). *If φ is a valid ancestral coloring, then for every real leaf $\ell \in \mathcal{L}$ and every color $c \in [m]$, the path $\text{Path}(\ell)$ contains at most one active node of color c .*

Proof. If there were two distinct nodes $u, v \in \text{Path}(\ell)$ with $\varphi(u) = \varphi(v) = c$, the definition of valid ancestral coloring would be violated.

Proposition VI.3 (Fixed-shape batch retrieval). *Assume a correct single-item PIR backend. Then any valid ancestral coloring lets the client recover all real proof nodes of a*

target leaf ℓ using one batch-PIR request with exactly m PIR subqueries.

Proof. By Proposition VI.2, each color contributes at most one real node to $\text{Path}(\ell)$. Hence the client retrieves at most one real item from each subdatabase, and uses a dummy query otherwise. The extracted real items are exactly the non-default proof material that must be supplied through PIR.

Proposition VI.4 (Correctness of default-hash completion). *Assume that the underlying single-item PIR scheme is correct. If φ is a valid ancestral coloring, then Algorithm 4 reconstructs the complete SMT proof of the target leaf ℓ from the real PIR outputs and the public default-empty hash chain.*

Proof. Fix a proof level $t \in \{0, \dots, h-1\}$. There are two cases. If the sibling subtree at level t contains at least one real leaf, then the corresponding proof node is active and belongs to $\text{Path}(\ell)$. By Proposition VI.2, it appears in exactly one color class, so Algorithm 3 records a unique non-dummy target τ_c , and correctness of PIR returns its digest in σ_c . Algorithm 4 then writes that digest into $P[t]$. Otherwise, the sibling subtree at level t is empty, and by SMT construction its digest is exactly the public default hash Δ_t , which remains in $P[t]$.

Therefore, every proof level is filled with the correct sibling digest, either from a real PIR output or from the public default-empty hash chain. Standard bottom-up Merkle verification using the path bits of ℓ then reconstructs the correct root digest.

Theorem VI.2 (Target privacy under setup leakage). *Assume the setup leakage profile and the target-privacy game in Definition IV.6. Assume also that the active color count m is polynomially bounded in the security parameter, as holds when the SMT height h is polynomially bounded because $m \leq h$. If each underlying single-item PIR query is query-private over its corresponding color subdatabase, and if dummy queries are sampled from a distribution computationally indistinguishable from well-formed PIR queries over that same subdatabase, then the batch retrieval protocol is target-private relative to $\mathcal{L}_{\text{setup}}(T)$.*

Proof. Fix any efficient adversary and any two target leaves ℓ_0, ℓ_1 chosen in the game. Condition on the setup leakage $\mathcal{L}_{\text{setup}}(T)$: the active metadata tables, color assignment, subdatabase sizes, active color count m , and the fact that every retrieval sends exactly one query to every active color are all already known. The only remaining transcript components are the m PIR queries themselves.

Consider one color c . For target ℓ_b , the client either has a unique real target index in $\mathcal{D}_c$ or has no real active node of that color and therefore sends a dummy query. If both targets induce real queries, query privacy of the underlying PIR makes the two selected indices indistinguishable. If one target induces a real query and the other induces a dummy query, the dummy-query assumption makes the dummy transcript indistinguishable from a well-formed PIR query over the same subdatabase, and query privacy hides the selected index. If

both induce dummy queries, the distributions are identical by construction.

Thus, for every color, the query transcript generated for ℓ_0 is computationally indistinguishable from the query transcript generated for ℓ_1 . Since m is polynomially bounded, a standard hybrid argument over the m colors shows that the entire batch transcript is computationally indistinguishable between $b = 0$ and $b = 1$. Hence every efficient adversary has negligible advantage in $\text{PrivTarget}_{\mathcal{A}}(\lambda)$.

Lemma VI.2 (Same-color intervals are disjoint). *If φ is a valid ancestral coloring, then for any color c , any two distinct intervals in*

$$\mathcal{I}_c = \{I_{\text{srv}}(u) : \varphi(u) = c\}$$

are disjoint.

Proof. Suppose two distinct nodes u, v satisfy $\varphi(u) = \varphi(v) = c$ and $I_{\text{srv}}(u) \cap I_{\text{srv}}(v) \neq \emptyset$. Then any real leaf $\ell \in I_{\text{srv}}(u) \cap I_{\text{srv}}(v)$ has both u and v on its proof path, contradicting path-distinctness.

Theorem VI.3 (Structural lower bound for active-color balance). *Let $N = |\mathcal{A}(T)|$ and let m be the active color count induced by the active-width model. For a node v in the interval forest, let $d(v)$ be the number of active nodes on the forest path from its root to v , including v , and let $\text{desc}(v)$ be the number of strict descendants of v in the interval forest. Define*

$$B(T) = \max \left\{ \left\lceil \frac{N}{m} \right\rceil, \max_{v: d(v) < m} \left\lceil \frac{\text{desc}(v)}{m - d(v)} \right\rceil \right\}.$$

Then every valid active coloring $\varphi : \mathcal{A}(T) \rightarrow [m]$ satisfies

$$\max_{c \in [m]} |\mathcal{D}_c| \geq B(T).$$

Proof. The term $\lceil N/m \rceil$ is the baseline equal-split lower bound. For the second term, fix any node v with $d(v) < m$. Along the active ancestor path from the forest root to v , validity forces $d(v)$ distinct colors to appear. Every strict descendant of v is also a descendant of all nodes on this path, and therefore cannot use any of these $d(v)$ colors. Hence all $\text{desc}(v)$ strict descendants must be assigned to at most $m - d(v)$ remaining colors. By the pigeonhole principle, at least one color subdatabase contains at least

$$\left\lceil \frac{\text{desc}(v)}{m - d(v)} \right\rceil$$

of these descendants. Taking the maximum over all such v and combining it with the equal-split bound proves the claim.

Proposition VI.5 (ActiveBalance certificate). *Let φ_{out} be the coloring returned by Algorithm 2, and let*

$$U = \max_{c \in [m]} |\mathcal{D}_c(\varphi_{\text{out}})|.$$

Then $U/B(T)$ is an instance-wise balance certificate: if OPT is the smallest possible largest subdatabase among all valid active colorings over $[m]$, then $U/\text{OPT} \leq U/B(T)$. Moreover, Algorithm 2 terminates at a subtree-permutation local optimum with respect to the lexicographic bucket signature Ψ .

Proof. Let OPT be the minimum possible value of $\max_c |\mathcal{D}_c|$ among all valid active colorings over $[m]$. By Theorem VI.3, $\text{OPT} \geq B(T)$. Therefore

$$\frac{U}{\text{OPT}} \leq \frac{U}{B(T)}.$$

This gives an a posteriori balance certificate for the returned instance.

For termination, observe that every accepted subtree permutation is required to strictly decrease the lexicographic signature Ψ , while rejected moves do not change the coloring. There are only finitely many valid active colorings of the active forest over $[m]$. Thus the algorithm cannot accept infinitely many moves. When it stops because no improving move is found, no candidate subtree color permutation considered by Algorithm 2 can further decrease Ψ , which is precisely subtree-permutation local optimality under this move set.

Theorem VI.4 (Indexing complexity). *Let $N = |\mathcal{A}(T)|$ be the total number of active proof-bearing nodes, and let $N_c = |\mathcal{D}_c|$ be the size of color subdatabase c . Then the interval-based preprocessing and query-indexing complexities satisfy*

$$\begin{aligned} \text{preprocessing time} &\in O(N \log N), \\ \text{preprocessing space} &\in O(N), \\ \text{single batch-PIR query indexing time} &\in O(m \log N). \end{aligned}$$

Proof. Computing the intervals of all active nodes takes $O(N)$ time. Sorting the interval tables costs

$$O\left(\sum_{c=1}^m N_c \log N_c\right) \subseteq O(N \log N).$$

At query time, the client performs one binary search per color, which costs

$$O\left(\sum_{c=1}^m \log N_c\right) \subseteq O(m \log N).$$

The total storage for active nodes and sorted interval tables is $O(N)$.

Theorem VI.5 (ActiveBalance complexity). *Fix one valid active coloring of an interval forest with N active nodes and active color count m . Then one execution of Algorithm 2 with at most R_1 accepted-or-rejected passes runs in*

$$O(R_1 N m^3)$$

time and uses $O(Nm)$ additional space.

Proof. In one pass, Algorithm 2 first annotates every subtree with its ancestor color mask and its per-color count histogram. Storing the count vectors for all N nodes costs $O(Nm)$ space, and computing them by one bottom-up traversal costs $O(Nm)$ time.

Now consider one fixed subtree root U . Its available-color set has size at most m , so building the local cost matrix K_U takes $O(m^2)$ time. The minimum-cost perfect matching on that square matrix is solved by the Hungarian algorithm in

$O(m^3)$ time. Forming the resulting candidate load vector and checking whether the global signature improves costs at most $O(m)$. Hence evaluating one subtree costs $O(m^3)$ time, and evaluating all N subtree roots in one pass costs $O(Nm^3)$.

The algorithm performs at most R_1 passes, and every accepted move strictly decreases the lexicographic signature $\Psi(\varphi)$, so no pass is repeated indefinitely. Therefore the total running time is $O(R_1 N m^3)$ and the peak extra space remains $O(Nm)$.

Theorem VI.6 (Client-side completion complexity). *Assume that the public metadata tables $\{\mathcal{M}_c\}_{c=1}^m$ are already available and that the default-empty hash chain $\Delta_0, \dots, \Delta_{h-1}$ has been precomputed once. Then Algorithm 4 runs in $O(h)$ time and uses $O(h)$ additional space. Consequently, the total online client work of query generation plus proof completion is*

$$O(m \log N + h).$$

Proof. Algorithm 4 initializes an array of length h , processes at most one extracted item per active color, and performs exactly h bottom-up hash combinations. Each step is constant-time apart from the hash evaluations, so the total running time is $O(h)$ and the extra space for the completed proof array is also $O(h)$. Combining this with the $O(m \log N)$ indexing time of Algorithm 3 yields the stated online complexity. The one-time public preprocessing of the default-empty hash chain costs $O(h)$ time and $O(h)$ space.

The following theorem states the end-to-end guarantee used in the evaluation: it ties the construction to correctness, target privacy, storage, metadata, and backend-facing cost.

Theorem VI.7 (End-to-end SparseTreePIR guarantee). *Fix a public height- h SMT snapshot T with active set $\mathcal{A}(T)$, active color count m , a valid coloring φ , color subdatabases $\mathcal{D}_1, \dots, \mathcal{D}_m$, and public metadata tables $\mathcal{M}_1, \dots, \mathcal{M}_m$. Assume that the underlying single-item PIR backend is correct and query-private for each subdatabase, and that dummy queries are sampled from the same query distribution as well-formed PIR queries over the corresponding subdatabase. Then SparseTreePIR satisfies the following end-to-end guarantee for every occupied target leaf:*

- 1) the client reconstructs a valid standard height- h SMT membership proof;
- 2) the retrieval is target-private relative to the setup leakage in Definition IV.5 and the target-privacy game in Definition IV.6;
- 3) the PIR database contains exactly $N = |\mathcal{A}(T)|$ digest records, and the public metadata size is $O(N)$;
- 4) client-side lookup and proof completion together cost $O(m \log N + h)$ time;
- 5) the backend workload is determined by the color-store load vector $\{|\mathcal{D}_c|\}_{c=1}^m$: a sequential execution performs one PIR query over each $\mathcal{D}_c$, while a parallel execution is bottlenecked by the largest per-color backend cost.

Proof. The first item follows from Proposition VI.3 and Proposition VI.4: each color returns at most one real active

sibling digest, and default-empty levels are filled locally. The second item is exactly Theorem VI.2. The PIR-facing database consists of one digest for each active proof-bearing node by construction, while the metadata tables store a constant number of fields per active node, so their size is $O(N)$. The fourth item combines Theorem VI.6 with the interval-search bound in Theorem VI.4. Finally, the online protocol issues one independent PIR subquery to each color subdatabase. Therefore backend work composes over the sizes of the searched color stores, with the sequential and parallel interpretations stated above.

Remark VI.2. *Algorithms 3 and 4 are directly implementable: all inputs required by proof completion are either public metadata, public default hashes, or client-local state already produced during query generation. No additional server round is needed after batch-PIR extraction.*

VII. EXPERIMENTAL EVALUATION

The evaluation studies SparseTreePIR as a private retrieval layout for fixed public SMT snapshots. We do not compare against ordinary SMT proof serving as if it were a privacy-preserving baseline: direct proof serving is included only as a semantic reference because it reveals the target. Instead, the experiments compare layouts that can be queried with a fixed-shape private transcript. The baselines are chosen to test the “simple sparse TreePIR extension” hypothesis directly: perfected TreePIR applies the full perfect-tree object, Pruned TreePIR- h deletes default records but keeps TreePIR’s height- h color/query universe, PBC-style SMT applies generic batch placement after active extraction, and flat active PIR removes inactive records but does not exploit active-width coloring. The evaluation asks four questions: (i) what PIR-facing database profile is induced by real SMT key sets; (ii) how much of the improvement comes from choosing the active proof object rather than a perfect-tree object; (iii) how much does ActiveBalance improve a valid active coloring; and (iv) whether the resulting layouts improve concrete executable PIR backends on the same real workloads.

A. Workloads, Platform, and Metrics

We use six SMT key sets. Five are address-derived account-key workloads from public Polygon zkEVM and ZKsync Era RPC data, and one is a FuelLabs SMT test-vector workload. The Polygon workloads expand observed addresses into the Polygon SMT key types for balance, nonce, code hash, and code-hash length. The recent Polygon workload covers blocks 32,046,547–32,096,546; the multi-window and broad variants sample four and eleven historical windows, respectively. The ZKsync sample and broad workloads cover blocks 70,082,262–70,132,261 and 69,632,271–70,132,270, expanding addresses into balance, nonce, code, and storage-root keys. The FuelLabs workload is generated from the `smt_test_spec.json` fixture in `FuelLabs/smt-test-generation`. These workloads are not full deployed state-tree dumps; they are reproducible account-key snapshots intended to stress the retrieval layouts

induced by realistic account-key distributions rather than to reproduce a complete archival state service. Exact source files, extraction scripts, block windows, and summary metadata are included in the artifact.

Each key set is mapped into height-128 and height-256 SMT coordinate spaces. Polygon and ZKsync keys are first hashed with SHA-256 and then mapped by prefix; FuelLabs vectors use their hexadecimal prefix coordinates. Duplicate input keys and duplicate coordinates after mapping are merged into one occupied SMT leaf. The main workload suite contains no synthetic random-leaf trees.

Structural metrics are deterministic functions of the mapped key set and the active interval forest: the active PIR record count $N = |\mathcal{A}(T)|$, the active batch width m , the largest color store $M(\varphi) = \max_c |\mathcal{D}_c|$, the dummy-slot fraction, and the public metadata size. These are the quantities exposed to the PIR layer. A backend whose cost depends on database size is driven by the color-store vector $(|\mathcal{D}_1|, \dots, |\mathcal{D}_m|)$: sequential work composes across colors, while parallel work is bottlenecked by the largest backend call.

Executable backend measurements were run on Ubuntu 22.04.5 with Linux 6.8, a 13th Gen Intel Core i5-13500HX CPU with 20 hardware threads, 15 GiB RAM, Python 3.10.12, Go 1.18.1, GCC/G++ 11.4, and CMake 3.22. The SimplePIR and PIANO checkouts are pinned at commits `e9020b03bf28` and `f7107cfadc15`, respectively; the artifact records the full hashes, environment, command lines, and seed-level CSVs. The machine does not expose AVX-512, so YPIR is not treated as an evaluated backend.

For backend measurements, we use the height-128 workloads and compare the PBC-style SMT baseline against SparseTreePIR. Each backend experiment is repeated with three target-sampling seeds, 73000, 83000, and 93000, and each seed requests 50 occupied targets per workload. Communication is deterministic once the layout and backend parameter tier are fixed, so the communication standard deviation is zero in the repeated runs; timing columns report the sample standard deviation over target-sampling seeds. Online communication counts backend upload and download bytes for the measured PIR calls. Setup time is backend preprocessing for the measured databases, while query time is client query generation only; server answering, local proof completion, one-time layout construction, and public metadata publication are excluded from those timing columns.

B. Privacy-Preserving Layouts

We compare five layout choices. *Perfected TreePIR* treats the SMT as a complete height- h perfect tree and is used only as an accounting baseline, because a height-128 or height-256 full coordinate tree is not materializable. *Pruned TreePIR- h* first removes public default records but keeps TreePIR’s height- h color/query universe; it is the direct “delete empty nodes” baseline. *PBC-style SMT* is an optimistic PBC-style accounting baseline over the same active proof records: it stores three replicated copies and spreads them over $\lceil 1.5m \rceil$ buckets. Our implementation uses deterministic balanced placement

rather than sampled Cuckoo placement when reporting bucket sizes. This gives the PBC-style baseline an optimistic bucket balance and removes seed dependence from the layout comparison. *Flat active PIR* stores active proof records once, but searches one global active database for each proof slot. *Sparse-TreePIR* stores the same active proof records once, partitions them by active-interval coloring, and applies ActiveBalance to reduce the largest color store.

a) *Pruned TreePIR- h baseline.*: Formally, this is a pruning-after-height baseline. For each verifier proof slot $i \in [h]$, let $\mathcal{P}_i$ be the set of non-default sibling digests that can appear in slot i across occupied-leaf membership proofs. The baseline removes public default-empty records from the searched payloads but keeps the original TreePIR-style height- h query transcript: online retrieval still sends one query slot per $i \in [h]$, using a dummy query when the target proof has no non-default sibling in that slot. Its width is therefore h , and its largest searched store is $\max_i |\mathcal{P}_i|$. This baseline tests whether deleting default records after committing to the height- h proof-query universe is sufficient; it does not re-identify the SMT-specific active proof-bearing interval forest or reduce the transcript to the active width m .

In Table IV, Active N is the number of PIR-facing proof records, m is the active batch width, Dummy is the fraction of dummy color slots under SparseTreePIR, Pruned width/max reports the TreePIR-style h -color query universe after deleting default records, PBC width/max reports $\lceil 1.5m \rceil$ buckets and the balanced PBC accounting bucket, and Sparse max is the largest SparseTreePIR color store after ActiveBalance. The table shows that the PIR-facing active object is small even when the verifier proof remains a height-128 or height-256 SMT proof. Across the workload suite, the active batch width is between 9 and 17, while the direct pruned-TreePIR baseline still issues 128 or 256 color queries. For the broad ZKsync workload, for example, 15478 active proof records are partitioned into 17 color stores and the largest contains 911 records, compared with a Pruned TreePIR-128 largest bucket of 3014 records. Increasing the logical height from 128 to 256 does not change these active records for the measured key sets because the occupied-leaf skeleton has already separated; the client still reconstructs the remaining verifier levels from public default hashes.

Table V separates the source of each improvement. Perfected TreePIR uses a proof object that is natural for complete trees but inflated for sparse SMT snapshots. Pruned TreePIR- h uses the active records but inherits TreePIR’s height-pinned h -color query universe, causing many dummy slots and level-bucket imbalance. PBC-style SMT also uses the active proof records, but treats them as an arbitrary batch and pays for replication and extra buckets. Flat active PIR removes inactive coordinates but still queries one large active database repeatedly. SparseTreePIR combines active-only storage, an active-width query pattern, and balanced color stores.

Overall, the evaluation isolates three effects. Active extraction removes the full coordinate tree from the PIR-facing database, avoiding a perfected object that is more than

$10^{34} \times$ larger even at height 128. Active-width coloring is more than pruning: it replaces the height- h pruned TreePIR query universe with the exact active width, giving $9.5 \times$ fewer subqueries and a $2.5 \times$ smaller largest pruned store on the height-128 real workloads. It also gives $1.5 \times$ fewer subqueries than the PBC-style baseline and a $12.7 \times$ smaller searched database than flat active PIR. ActiveBalance then reduces the largest searched color store by another $1.2 \times$ without changing the active width. These effects are orthogonal to the choice of PIR backend: the backend experiments below measure whether the improved layout profile carries over to existing PIR implementations.

C. Effect of ActiveBalance

A valid active coloring is not automatically a good backend layout. Table VI compares ActiveBalance with a node-local hybrid initializer on the height-128 workloads. The node-local initializer greedily assigns feasible colors using ancestor-color constraints and current global color loads, but it does not apply subtree-level color permutations. Both schemes use the same active records and the same active width; the only difference is the balancing refinement. The lower column is the structural lower bound $B(T)$ from Theorem VI.3.

In five of the six height-128 workloads, ActiveBalance reaches the reported structural lower bound exactly; in the Fuel test-vector workload it is within $1.045 \times$ of the lower bound. The metadata column uses the prototype encoding of 24 public bytes per active record, compared with 32 private digest bytes per active record. Thus the public lookup state is linear in the number of active proof records and remains far from the full coordinate-space scale.

D. Executable PIR Backends on Real Workloads

We next measure concrete backend behavior. The SimplePIR integration materializes the 32-byte digest records for each color store and calls the official SimplePIR Go API [36], including Setup, Query, Answer, and Recover. Each digest is retrieved as eight 32-bit chunks because the artifact’s convenient database constructor represents one entry as a machine word. The bridge verifies recovered chunks, so this is an end-to-end digest-retrieval check, although it is not optimized digest packing.

The PIANO integration uses the public PIANO artifact [37] with the exact bucket-size vector induced by each real workload layout. It is therefore an executable backend-shape runner rather than a full SMT digest-reconstruction bridge. PIANO’s artifact internally caps the number of subdatabase queries according to its refresh window for small databases; the raw tables record the completed subdatabase-level query counts. Accordingly, we treat PIANO as a secondary backend-shape check, while the SimplePIR bridge is the main end-to-end digest-retrieval backend.

Table VII is the main backend result. Against the PBC-style SMT baseline, SparseTreePIR gives $2.1 \times$ lower online communication under the full SimplePIR bridge and $2.3 \times$ lower online communication under the PIANO runner. In the

TABLE IV
SMT WORKLOAD PROFILES AND INDUCED PRIVACY-PRESERVING LAYOUT COSTS.

Workload	h	Keys	Active N	m	Avg. path	Dummy	Pruned width/max	PBC width/max	Flat max	Sparse max
FuelLabs SMT vectors	128	100	198	9	6.88	23.56%	128 / 40	14 / 43	198	23
FuelLabs SMT vectors	256	100	198	9	6.88	23.56%	256 / 40	14 / 43	198	23
Polygon recent	128	384	766	11	8.89	19.15%	128 / 168	17 / 136	766	70
Polygon recent	256	384	766	11	8.89	19.15%	256 / 168	17 / 136	766	70
ZKsync sample	128	956	1910	13	10.19	21.62%	128 / 370	20 / 287	1910	147
ZKsync sample	256	956	1910	13	10.19	21.62%	256 / 370	20 / 287	1910	147
Polygon multi-window	128	1348	2694	14	10.73	23.36%	128 / 530	21 / 385	2694	193
Polygon multi-window	256	1348	2694	14	10.73	23.36%	256 / 530	21 / 385	2694	193
Polygon broad	128	7040	14078	17	13.11	22.86%	128 / 2762	26 / 1625	14078	829
Polygon broad	256	7040	14078	17	13.11	22.86%	256 / 2762	26 / 1625	14078	829
ZKsync broad	128	7740	15478	17	13.25	22.03%	128 / 3014	26 / 1786	15478	911
ZKsync broad	256	7740	15478	17	13.25	22.03%	256 / 3014	26 / 1786	15478	911

TABLE V
MEASURED SOURCES OF IMPROVEMENT ON THE REAL SMT WORKLOAD SUITE. MULTIPLICATIVE GAINS ARE COMPUTED FROM ARITHMETIC-MEAN REDUCTIONS OVER THE SIX HEIGHT-128 WORKLOAD INSTANCES.

Comparison	Observed resource effect	Interpretation
vs. perfected TreePIR	Full $h = 128/256$ objects are $> 4.3 \cdot 10^{34} \times / > 10^{73} \times$ larger than the largest active object	Full coordinate trees are a mismatched PIR-facing object for real SMT workloads.
vs. Pruned TreePIR- h	$9.5 \times$ fewer subqueries; $2.5 \times$ smaller largest pruned store	Pruning default records still leaves the wrong height- h query universe.
vs. PBC-style SMT	$1.5 \times$ fewer subqueries; $1.9 \times$ smaller largest bucket	Generic batch placement does not exploit active interval/path structure.
vs. flat active PIR	$12.7 \times$ smaller max searched database	Active storage alone is insufficient; color-store splitting changes the backend profile.
vs. node-local active coloring	$1.2 \times$ smaller max color store	ActiveBalance improves the exact-width active color-store profile.

TABLE VI
ACTIVEBALANCE ABLATION ON REAL HEIGHT-128 ACTIVE INTERVAL FORESTS

Workload	Active N	m	Lower	Hybrid max	AB max	Red.	Meta KB
FuelLabs SMT vectors	198	9	22	26	23	11.5%	4.6
Polygon recent	766	11	70	83	70	15.7%	18.0
ZKsync sample	1910	13	147	180	147	18.3%	44.8
Polygon multi-window	2694	14	193	236	193	18.2%	63.1
Polygon broad	14078	17	829	929	829	10.8%	330.0
ZKsync broad	15478	17	911	1162	911	21.6%	362.8

TABLE VII
REPEATED EXECUTABLE-BACKEND RESULTS ON REAL HEIGHT-128 WORKLOADS. EACH ENTRY IS A BASELINE/SPARSE TREEPIR COST RATIO AVERAGED OVER SIX WORKLOADS, THREE SEEDS, AND 50 REQUESTED OCCUPIED TARGETS PER SEED. ANSWER GAIN IS SIMPLEPIR SERVER-ANSWER TIME; SETUP GAIN IS BACKEND PREPROCESSING ONLY AND EXCLUDES LAYOUT CONSTRUCTION AND METADATA PUBLICATION.

Backend	Runs	Width gain	Max-bucket gain	Online gain	Query gain	Answer gain	Setup gain
SimplePIR full API	3×50	$1.5 \times$	—	$2.1 \times$	$1.7 \times$	$1.8 \times$	$2.3 \times$
PIANO local runner	3×50	$1.5 \times$	$1.9 \times$	$2.3 \times$	$2.4 \times$	—	$3.1 \times$

SimplePIR full-API bridge, server-answer time is $1.8 \times$ lower on average; PIANO answer timers are not reported because the artifact’s small-database counters are often zero or quantized.

TABLE VIII
RAW SIMPLEPIR FULL-API MEANS BEHIND TABLE VII. VALUES ARE ARITHMETIC MEANS OVER THE THREE TARGET-SAMPLING SEEDS; COMMUNICATION IS KB PER REQUESTED PROOF AND TIMINGS ARE MILLISECONDS.

Workload	PBC KB	Sparse KB	PBC query	Sparse query	PBC answer	Sparse answer
Fuel vectors	12,000	5,906	9,061	5,936	0.033	0.023
Polygon recent	25,500	11,688	12,180	7,609	0.044	0.029
ZKsync sample	43,125	20,719	15,817	9,684	0.076	0.038
Polygon multi-window	51,844	25,375	17,792	10,633	0.081	0.047
Polygon broad	133,250	62,156	32,367	17,223	0.245	0.114
ZKsync broad	138,938	65,344	32,302	17,679	0.268	0.120

TABLE IX
PER-WORKLOAD EXECUTABLE-BACKEND GAINS AGAINST THE PBC-STYLE SMT BASELINE. WIDTH REPORTS PBC-STYLE BUCKETS / SPARSE TREEPIR COLORS; OTHER ENTRIES ARE BASELINE/SPARSE TREEPIR COST RATIOS AVERAGED OVER THREE SEEDS.

Workload	Width	SimplePIR online	SimplePIR query	PIANO online	PIANO query	PIANO max-bucket
FuelLabs SMT vectors	14 / 9	$2.0 \times$	$1.5 \times$	$3.2 \times$	$3.4 \times$	$1.9 \times$
Polygon recent	17 / 11	$2.2 \times$	$1.6 \times$	$3.1 \times$	$2.7 \times$	$1.9 \times$
ZKsync sample	20 / 13	$2.1 \times$	$1.6 \times$	$2.1 \times$	$1.8 \times$	$2.0 \times$
Polygon multi-window	21 / 14	$2.0 \times$	$1.7 \times$	$2.1 \times$	$2.1 \times$	$2.0 \times$
Polygon broad	26 / 17	$2.1 \times$	$1.9 \times$	$2.1 \times$	$3.1 \times$	$2.0 \times$
ZKsync broad	26 / 17	$2.1 \times$	$1.8 \times$	$2.1 \times$	$2.1 \times$	$2.0 \times$

Query generation and backend setup time also improve by $1.7 \times$ – $3.1 \times$. The ratio table is backed by checked-in raw seed-level CSVs: each row records the baseline and SparseTreePIR online KB, query-generation time, SimplePIR server-answer time, and backend setup time for a workload and target-sampling seed. These gains are not caused by changing the PIR primitive; both schemes are measured through the same backend in each row. They come from the database profile exposed to the backend: the PBC-style baseline stores three copies over approximately $1.5m$ buckets, while SparseTreePIR stores one copy over m active color stores.

Table VIII gives the absolute SimplePIR values behind the aggregate ratios. The ratios in Table VII are computed from per-workload reductions, so they need not equal ratios of rounded arithmetic means in the raw table. The artifact also includes an aggregate raw-means CSV for both SimplePIR and PIANO, including backend setup time.

Table IX shows that the communication improvement is stable across workloads. SimplePIR online communication is $2.0 \times$ – $2.2 \times$ lower, and PIANO online communication is $2.1 \times$ – $3.2 \times$ lower. Timing gains vary more than communication

because they depend on backend parameter tiers, artifact-local refresh behavior, and small-database effects; the SimplePIR answer-time values are especially small and should be read as corroborating evidence rather than as the primary performance metric. The deterministic communication and database-profile metrics are the more stable backend-facing evidence, and every workload still benefits in the repeated measurements. These results support the intended experimental claim: the active-interval layout improves the concrete PIR workload seen by existing backends.

E. Reproducibility and Scope

All tables in this section are generated from repository scripts and checked-in CSV outputs. The repeated backend run writes per-seed raw tables, per-seed summaries, an aggregate mean/stddev table, and an environment record. The structural tables are deterministic for the input key sets; the backend timing tables are repeated because artifact-level timings are machine-dependent. Communication is reported as a backend-facing byte count for the measured layout and does not include one-time publication of public metadata.

The scope of the evidence is intentionally narrow. The SimplePIR bridge and PIANO runner are the concrete backend integrations used as main evidence. The local LWE-style prototype, two-server XOR model, and public TreePIR artifact smoke tests are useful for development and appendix-level context, but they are not used as the primary backend claims in this section. In particular, the TreePIR VBPIR and Spiral artifacts execute on TreePIR-format perfect-tree layouts; a full SparseTreePIR integration would require exporting active color stores into each backend-specific database encoding and validating extraction. YPIR is not run on this platform because the machine lacks AVX-512 support, and SealPIR-style service integration requires a heavier SEAL, Protobuf, and gRPC build stack. These are future integration targets rather than unreported completed experiments.

The protocol scope is also fixed. We evaluate membership-proof retrieval for occupied leaves in a fixed public SMT snapshot. Non-membership retrieval over absent coordinates would require a different served-interval domain, a different leakage model, and explicit treatment of whether the present-versus-absent query type is hidden. Dynamic updates would likewise require an epoch-level leakage and amortization analysis. Those extensions are separate from the static active-interval retrieval layer validated here.

VIII. CONCLUSION AND FUTURE DIRECTIONS

We introduced SparseTreePIR, a direct active-interval layout for fixed binary SMT membership-proof retrieval. SparseTreePIR preserves the standard height- h verifier proof semantics while making the PIR-facing database consist only of active proof-bearing sibling digests. Served-interval metadata, fixed-shape real/dummy PIR queries, and public default-hash completion together define a complete private retrieval layer for this SMT-specific object.

The analysis shows that the active-node, one-query-per-color model is governed by active proof material rather than by every logical SMT level. The algorithmic contribution is that a valid active coloring is not sufficient by itself: color subdatabases must also be balanced. The subtree-level ActiveBalance refinement keeps the active query pattern fixed while moving the largest bucket close to the structural lower bound across the tested real SMT workloads. The experiments compare privacy-preserving layout choices over the same active proof material: direct active-interval coloring improves the relevant resource profile through active-only storage, active color count m , and a flatter largest searched subdatabase. The SimplePIR and PIANO bridges demonstrate compatibility with existing PIR implementations and show substantial improvement over flat uncolored-active PIR and the PBC-style SMT baseline.

The main extensions remain SMT-centered. First, non-membership SparseTreePIR should extend the served-interval abstraction from occupied-leaf ranks to absent logical-coordinate regions and define whether query existence is hidden. Second, the retrieval layer should be integrated with deployed sparse authenticated stores such as Revocation Transparency-style maps and Jellyfish Merkle Tree [11], [17]. Third, the instance-wise ActiveBalance certificate should be strengthened into an a priori guarantee for broader classes of interval forests.

REFERENCES

- [1] R. C. Merkle, "A Certified Digital Signature," in *Advances in Cryptology—CRYPTO '89 Proceedings*, G. Brassard, Ed., LNCS 435, Springer, 1990, pp. 218–238.
- [2] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency Version 2.0," RFC 9162, Internet Engineering Task Force, Dec. 2021.
- [3] B. Laurie, "Certificate Transparency," *ACM Queue*, vol. 12, no. 8, pp. 10–19, 2014.
- [4] S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging," in *Proc. 18th USENIX Security Symposium (USENIX Security)*, 2009.
- [5] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's Highly Available Key-Value Store," in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205–220.
- [6] A. Anagnostopoulos, M. T. Goodrich, and R. Tamassia, "Persistent Authenticated Dictionaries and Their Applications," in *Information Security*, LNCS 2200, Springer, 2001, pp. 379–393.
- [7] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, "CONIKS: Bringing Key Transparency to End Users," in *Proc. 24th USENIX Security Symposium (USENIX Security)*, 2015, pp. 383–398.
- [8] I. Tzialla, A. Kothapalli, B. Parno, and S. Setty, "Transparency Dictionaries with Succinct Proofs of Correct Operation," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] H. Malvai, L. Kokoris-Kogias, A. Sonnino, E. Ghosh, E. Ozturk, K. Lewi, and S. F. Lawlor, "Parakeet: Practical Key Transparency for End-to-End Encrypted Messaging," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2023.
- [10] J. Len, M. Chase, E. Ghosh, K. Laine, and R. Cruz Moreno, "OPTIKS: An Optimized Key Transparency System," in *Proc. 33rd USENIX Security Symposium (USENIX Security)*, 2024, pp. 4355–4372.
- [11] B. Laurie and E. Kasper, "Revocation Transparency," Google Research technical report, Sep. 2012.
- [12] D. Koisser, P. Jauernig, G. Tsudik, and A.-R. Sadeghi, "V'CER: Efficient Certificate Validation in Constrained Networks," in *Proc. 31st USENIX Security Symposium (USENIX Security)*, 2022, pp. 4491–4508.

- [13] B. Bailey and S. Sankagiri, "Merkle Trees Optimized for Stateless Clients in Bitcoin," in *Financial Cryptography and Data Security*, LNCS 12676, Springer, 2021, pp. 451–466.
- [14] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Project Yellow Paper, 2014.
- [15] D. E. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash Protocol Specification," Version 2023.4.0, 2023.
- [16] R. Dahlberg, T. Pulls, and R. Peeters, "Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-)Membership Proofs," *Cryptology ePrint Archive*, Paper 2016/683, 2016.
- [17] Z. Gao, Y. Hu, and Q. Wu, "Jellyfish Merkle Tree," Diem Association technical paper, revised Jan. 12, 2021.
- [18] L. Ramabaja and A. Avdullahu, "Compact Merkle Multiproofs," arXiv preprint arXiv:2002.07648, 2020.
- [19] B. Ma, V. N. Pathak, L. Liu, and S. Ruj, "One-Phase Batch Update on Sparse Merkle Trees for Rollups," arXiv preprint arXiv:2310.13328, 2023.
- [20] M. I. Silva, J. Messias, and B. Livshits, "A Public Dataset For the ZKsync Rollup," arXiv preprint arXiv:2407.18699, 2024.
- [21] M. Petkus, "Efficient (Non-)Membership Tree from Multicollision-Resistance with Applications to Zero-Knowledge Proofs," *Cryptology ePrint Archive*, Paper 2024/1259, 2024.
- [22] Q. Cao, S. H. Dau, R. Gagiano, D. Huynh, X. Yi, P. L. Le, Q.-H. Luu, E. Viterbo, Y.-C. Huang, J. Zhu, M. M. Jalalzai, and C. Feng, "TreePIR: Efficient Private Retrieval of Merkle Proofs via Tree Colorings with Fast Indexing and Zero Storage Overhead," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2025, pp. 4476–4494.
- [23] S. Eskandarian, E. Messeri, J. Bonneau, and D. Boneh, "Certificate Transparency with Privacy," *Proc. Privacy Enhancing Technologies*, vol. 2017, no. 4, pp. 329–344, 2017.
- [24] D. Kales, O. Omolola, and S. Ramacher, "Revisiting User Privacy for Certificate Transparency," in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, 2019, pp. 432–447.
- [25] W. Lueks and I. Goldberg, "Sublinear Scaling for Multi-Client Private Information Retrieval," in *Financial Cryptography and Data Security*, LNCS 8975, Springer, 2015, pp. 168–186.
- [26] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, "Private information retrieval," *Journal of the ACM*, vol. 45, no. 6, pp. 965–981, 1998.
- [27] E. Kushilevitz and R. Ostrovsky, "Replication Is Not Needed: Single Database, Computationally-Private Information Retrieval," in *Proc. 38th Annu. IEEE Symp. Foundations of Computer Science (FOCS)*, 1997, pp. 364–373.
- [28] C. Cachin, S. Micali, and M. Stadler, "Computationally Private Information Retrieval with Polylogarithmic Communication," in *Advances in Cryptology—EUROCRYPT '99*, LNCS 1592, Springer, 1999, pp. 402–414.
- [29] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, "Batch Codes and Their Applications," in *Proc. 36th Annu. ACM Symp. Theory of Computing (STOC)*, 2004, pp. 262–271.
- [30] D. R. Stinson, R. Wei, and M. B. Paterson, "Combinatorial Batch Codes," *Advances in Mathematics of Communications*, vol. 3, no. 1, pp. 13–27, 2009.
- [31] R. Pagh and F. F. Rodler, "Cuckoo Hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [32] C. Aguilar-Melchor, J. Barrier, L. Fousse, and M.-O. Killijian, "XPIR: Private Information Retrieval for Everyone," *Proc. Priv. Enhancing Technol.*, vol. 2016, no. 2, pp. 155–174, 2016.
- [33] S. Angel, H. Chen, K. Laine, and S. Setty, "PIR with Compressed Queries and Amortized Query Processing," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2018, pp. 962–979.
- [34] M. H. Mughees, H. Chen, and L. Ren, "OnionPIR: Response Efficient Single-Server PIR," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, pp. 2292–2306.
- [35] S. J. Menon and D. J. Wu, "Spiral: Fast, High-Rate Single-Server PIR via FHE Composition," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2022, pp. 930–947.
- [36] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, "One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval," in *Proc. 32nd USENIX Security Symposium (USENIX Security)*, 2023, pp. 3889–3905.
- [37] M. Zhou, A. Park, W. Zheng, and E. Shi, "Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 4296–4314.
- [38] S. J. Menon and D. J. Wu, "YPIR: High-Throughput Single-Server PIR with Silent Preprocessing," in *Proc. 33rd USENIX Security Symposium (USENIX Security)*, 2024, pp. 5985–6002.
- [39] M. H. Mughees and L. Ren, "Vectorized Batch Private Information Retrieval," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2023, pp. 437–452.
- [40] J. Liu, J. Li, D. Wu, and K. Ren, "PIRANA: Faster Multi-query PIR via Constant-weight Codes," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 4315–4330.
- [41] A. Miller, M. Hicks, J. Katz, and E. Shi, "Authenticated Data Structures, Generically," in *Proc. 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2014, pp. 411–423.
- [42] T. Pulls and R. Peeters, "Balloon: A Forward-Secure Append-Only Persistent Authenticated Data Structure," in *Computer Security—ESORICS 2015*, LNCS 9326, Springer, 2015, pp. 622–641.